



Quality assurance of LLM-generated code: Addressing non-functional quality characteristics[☆]

Xin Sun^{ID}*, Daniel Ståhl^{ID}, Kristian Sandahl^{ID}, Christoph Kessler^{ID}

Department of Computer and Information Science, Linköping University, Sweden

ARTICLE INFO

Keywords:

LLM-generated code
Code generation
Non-functional quality characteristics
Maintainability
Security
Performance efficiency

ABSTRACT

In recent years, large language models have been widely integrated into software engineering workflows, supporting tasks like code generation. While prior evaluations focus on functional correctness, there is still a limited understanding of the non-functional quality characteristics of generated code.

Guided by the ISO/IEC 25010 quality model, this study adopts a multi-methods approach comprising three complementary elements: a literature review of 109 papers, two industry workshops with practitioners from multiple organizations, and an empirical analysis of patching real-world software issues using three LLMs. Motivated by insights from both the literature and practitioners, the empirical study examined the quality of generated patches regarding security, maintainability, and performance efficiency, which were identified as critical code-level quality attributes.

Our results indicate that existing research primarily emphasizes security, performance efficiency, and maintainability, while other quality attributes are understudied. In contrast, practitioners prioritize maintainability and readability, warning that generated code may accelerate the accumulation of technical debt. The empirical evaluation demonstrates the instability of optimizing NFQCs through prompts in practical software engineering settings.

Overall, our findings expose a misalignment between academic focus, industry priorities, and observed model behavior, highlighting the need to integrate quality assurance mechanisms into LLM code generation pipelines to ensure that future generated code not only *passes tests* but truly *passes with quality*.

1. Introduction

The prevalence of large language models for code (LLMs for code) has brought new changes to the field of software engineering (Chen et al., 2021; Rozière et al., 2023). These models are tailored for understanding and processing code and can generate functionally correct outputs under appropriate prompts (Luo et al., 2024; Hui et al., 2024; Jiang et al., 2023). Currently, these powerful models have been integrated into the workflow of software engineering and are used by millions of developers worldwide. GitHub Copilot, released by GitHub and OpenAI, is an AI pair programmer that is built on Codex, which is a family of code LLMs (Chen et al., 2021; GitHub, 2025). Copilot can generate code across multiple programming languages from various types of prompts, such as natural language descriptions, function signatures, and surrounding code. In some cases, it can produce complete applications such as interactive websites or data pipelines from a single prompt. As these models continue to evolve rapidly, their potential and

implications for software engineering remain in flux (The AI Digest, 2024).

Several carefully curated metrics and benchmarks have been developed to evaluate the generation capability of code LLMs (Chen et al., 2021; Jimenez et al., 2024; Zhang et al., 2024; Deng et al., 2025; Li et al., 2025; Du et al., 2024). Chen et al. (2021) released HumanEval, together with GitHub Copilot, to evaluate the ability of code LLMs to generate functionally correct code. SWE-bench (Jimenez et al., 2024) was introduced to assess the ability of LLMs to solve real-world software engineering tasks.

While these benchmarks provide a good basis for evaluating the ability of LLMs to generate functionally correct code, they offer limited support for assessing the non-functional quality characteristics (NFQCs) of generated code. According to the ISO/IEC 25010 quality model (International Organization for Standardization, 2023), software quality encompasses not only functional correctness but also NFQCs such as performance efficiency, maintainability, and security.

[☆] Editor: Heiko Koziulek.

* Corresponding author.

E-mail addresses: xin.sun@liu.se (X. Sun), daniel.stahl@liu.se (D. Ståhl), kristian.sandahl@liu.se (K. Sandahl), christoph.kessler@liu.se (C. Kessler).

¹ In this study, **generated code** refers to source code generated by LLMs, including both code snippets and code patches.

Due to the limited assessments of NFQCs in the current evaluation system, code LLMs may produce code with varying quality along different NFQCs, which makes the generated code¹ unreliable and sometimes causes severe faults. For example, Pearce et al. (2022) studied the performance of GitHub Copilot in high-risk cybersecurity scenarios (MITRE, 2024). In their study of 1689 programs generated across 89 scenarios, approximately 40% were found to contain code patterns considered vulnerable, illustrating the security weakness in generated code.

In particular, it remains unclear how NFQCs of generated code are currently conceptualized and assessed in existing research, whether these studies align with practitioner concerns, and how they interact with each other in concrete code generation settings beyond functional correctness.

To address our research goal, we adopt a mixed-method design in which each component plays a distinct role, while using the ISO/IEC 25010 quality model as a common reference to structure and relate the investigation of NFQCs.

First, the literature review examines how NFQCs of generated code are currently conceptualized and evaluated in existing studies, revealing both commonly studied attributes and gaps. Secondly, the industry workshops provide an independent perspective on NFQCs by capturing how practitioners perceive, prioritize, and reason about quality concerns of generated code in real-world development settings. This step allows us to contrast research evaluation practices with practitioner expectations, rather than assuming their alignment. Finally, informed by insights from both the literature and the workshops, the empirical study focuses on a set of NFQCs that are consistently highlighted as important across research and practice. Rather than evaluating each quality characteristic in isolation, the study examines how multiple NFQCs are jointly reflected in SWE-bench under different evaluation and optimization choices. This design enables us to explore patterns for trade-offs among selected NFQCs and to illustrate how current evaluation practices may influence NFQCs when multiple quality concerns are considered simultaneously.

Together, the mixed-methods form a complementary design that connects conceptual definitions, practitioners' perspectives, and empirical observations, providing an integrated perspective on NFQCs in generated code.

To guide our study, we defined three overarching **Research Goals (RGs)**, each addressing a different aspect. The research goals are as follows:

- **RG1:** Explore how NFQCs in generated code have been addressed in existing research, and identify research gaps in this area.
- **RG2:** Identify the real-world expectations and pain points that practitioners have when integrating generated code in software projects.
- **RG3:** Characterize the observed behavior of generated code across key NFQCs identified in the previous research goals, and examine how feedback-driven strategies affect these qualities and the trade-offs among them in a real-world software engineering context.

For each research goal, we further formulated corresponding **Research Questions (RQs)** to guide the analysis, which are presented in the corresponding sections.

The rest of the paper is organized as follows: Section 2 introduces the background on code generation and related work. Section 3 outlines the structure of the literature review. Section 4 presents the organization of the two industry workshops, detailing the participant selection, session design, and experience of practitioners. Section 5 summarizes the setup of the empirical evaluation, covering dataset selection, experiment configuration, evaluation metrics, and analysis workflow. Additionally, we discuss the implications of our findings, highlighting challenges and opportunities for future research. We discuss the threats to validity in Section 6. Finally, Section 7 concludes the study and outlines directions for integrating NFQCs into the quality assurance of LLM-generated code in real-world software systems.

2. Background and related work

This section presents the background of LLMs for code generation and related works.

2.1. Code generation

Code generation has become a more important application scenario since LLMs were first proposed (Manna and Waldinger, 1971; Liventsev et al., 2023). Code generation aims to automatically generate code given the problem description, high-level specifications, or existing code. Before the advent of LLMs, code generation was limited to structured methods (Hemel et al., 2008). These methods usually require detailed formal specifications and several steps for generation. LLMs changed this paradigm by learning from large code corpora, thus eliminating the need for formal specifications and allowing code generation from natural language prompts (Luo et al., 2024).

The release of GitHub Copilot marked the beginning of integrating LLMs into daily software workflows. Since then, numerous code LLMs have emerged, demonstrating remarkable performance in generating functionally correct and meaningful code across various programming languages (Rozière et al., 2023; Hui et al., 2024; Jiang et al., 2023; Chen et al., 2021; Li et al., 2022, 2023).

By 2025, many AI programming tools have emerged, and a new approach called *vibe coding* has gained prominence (Lovable, 2024; Bolt, 2024; Cursor, 2024; Windsurf, 2024). In *vibe coding*, users do not directly engage with the code; instead, they remain entirely at the level of natural language, formulating prompts that instruct the code LLMs to generate, modify, and deploy components of the software. For example, Lovable² is an AI-driven platform that enables users to build full-stack applications using natural language prompts, allowing rapid prototyping and deployment without extensive coding knowledge. These advancements have significantly lowered the barrier to programming and accelerated software development processes.

2.2. Literature reviews

A review by Wang and Chen (2023) examined LLM code generation from a different angle. By reviewing 20 existing studies, they demonstrated that while using LLMs to generate code has been widely studied, the evaluation of LLM-generated code had received relatively little attention. The study addressed functional requirements such as functional correctness, as well as NFQCs, including security, maintainability, and others. It also highlighted key limitations in current evaluation practices, such as the use of inadequate evaluation criteria, the absence of systematic and quantitative evaluation frameworks, and the lack of consideration for human involvement in the evaluation process.

Z. Yang et al. (2024) presented a systematic literature review of 146 studies focusing on NFQCs of code LLMs. They identified and discussed six NFQCs beyond functional correctness: *robustness*, *security*, *privacy*, *explainability*, *efficiency* and *usability*. The study highlighted the vulnerability of LLM4Code systems to adversarial attacks, data poisoning, and privacy leaks, as well as challenges in explainability and usability. To address these issues, they proposed three complementary perspectives, which are data-centric, human-centric, and system-centric, for developing more reliable and effective LLM4Code systems in the future. This work provided a comprehensive overview for understanding the broader implications of adopting LLM4Code in software engineering. However, most of the studies included in Z. Yang et al. (2024)'s literature review focus on evaluating the models themselves, such as the architecture, training data, or prompting strategies, rather than the quality of the code generated by LLMs. In practice, a model capable of

² <https://lovable.dev/>

producing high-quality code in principle does not guarantee that the actual outputs will meet high standards across NFQCs. It is therefore essential to acknowledge that, in real-world practice, systematic evaluation of generated code is required to ensure the overall quality of the software system. This motivates our study, which differs from prior work by shifting the perspective from the model to its outputs, and by examining to what extent the generated code satisfies NFQCs.

2.3. Empirical studies

Beyond conceptual discussions and literature reviews, a growing body of studies has examined the NFQCs of LLM-generated code. Niu et al. (2024a) conducted a comprehensive performance study using HumanEval and MBPP benchmarks and a set of programming problems from LeetCode, an online programming practice platform. They also investigated generating efficient code using prompt engineering methods. Their findings noted that the performance can depend heavily on how a prompt is given. If not instructed, LLMs might output a straightforward solution that ensures correctness but is not efficient.

Fu et al. (2025a) found that approximately 27.3% of generated code contained security weaknesses, spanning 43 Common Weakness Enumeration categories, including critical issues like Insufficiently Random Values and Cross-site Scripting. The study also explored the effectiveness of Copilot in fixing these vulnerabilities, demonstrating that enhanced prompts could resolve up to 55.5% of issues.

In the area of maintainability, Liu et al. (2024a) applied static analysis tools such as Pylint to assess generated code and reported frequent code smells and issues, suggesting the need for human oversight.

Taken together, prior work reveals a growing effort towards understanding the NFQCs of generated code. Our study extends that effort by integrating evidence from research, practice, and empirical evaluation.

3. Literature review

To be able to achieve RG1 in Section 1, we first conducted a literature review to assess the current state of research and potential issues in this field. Furthermore, the literature review sought to answer the following research questions:

- **RQ 3.1:** What trends can be observed in research on NFQCs of generated code?
- **RQ 3.2:** How have different NFQCs been examined in the existing studies, and what challenges and limitations have been identified?

3.1. Research design

LLMs have been utilized across various tasks. In this literature review, we focus especially on code generation. Code generation involves producing source code from natural language descriptions or limited hints, allowing NFQCs to emerge directly from the generation process. As this form of AI assistance is widely used in practice, studying it provides a representative context for the quality of generated code.

The search process of the literature review combines a structured search strategy, clearly defined inclusion and exclusion criteria, and the use of snowballing to identify additional relevant literature. We included papers that examine code LLMs and AI programming tools or platforms that integrate these models, such as GitHub Copilot. To be eligible, the selected papers must explore the NFQCs of the code generated by these tools or models. To facilitate consistent discussion and comparison, based on the ISO/IEC 25010 quality model, we map the diverse quality characteristics mentioned across studies to their corresponding terms in the ISO/IEC 25010 software product quality model, thereby standardizing terminology for our analysis. The search date is January 20th, 2026.

Table 1
The selection criteria of the initial search.

Inclusion criteria
• The paper discusses one of the NFQCs or similar quality characteristics of LLM-generated code. • The paper proposes an approach, study, or benchmark to evaluate one of the non-functional quality attributes of LLM-generated code. • The paper proposes new metrics to evaluate one of the non-functional quality attributes of LLM-generated code.
Exclusion criteria
• Paper that is not written in English. • Paper that is not within the subject of Computer Science and Engineering. • Paper that appeared before 2022.

Search strategy. We conducted the search on Scopus, chosen for its extensive coverage of peer-reviewed publications and inclusion of some preprints. Specifically, we limited our search to studies published between Jan. 2022 and Dec. 2025, as the majority of studies in code generation prior to 2022 were based on traditional approaches rather than LLMs, which gained major traction with the release of ChatGPT in late 2022. Our search query was formulated in two stages. First, we identified three core concepts: (1) LLMs, (2) code generation, and (3) software quality. Next, we broadened the first and third concepts using closely related terms to maximize recall while maintaining precision. For example, for LLMs, we included terms like ‘AI’ and ‘artificial intelligence’; for quality, we included ‘performance’ and ‘non-functional’. To ensure the included papers focus on code generation, we kept ‘code generation’ unchanged.

- LLM; large language model; AI; artificial intelligence
- code generation
- quality; performance; non-functional

To ensure a high coverage of relevant papers, we deliberately avoided overly specific keywords related to individual NFQCs (e.g., “robust”, “efficient”, “latency”) in the keywords list, as these terms can vary widely across papers and might appear in different forms. Finally, we set our search query as follows:

(“LLM” OR “large language models” OR “AI” OR “Artificial intelligence”) AND (“code generation”) AND (“quality” OR “performance” OR “non-functional”)

The initial results contained many irrelevant studies, so we applied a series of filters to refine the search. We also restricted the subject area to computer science and Engineering, as these two categories in Scopus most accurately capture research related to LLMs and code generation. The language is restricted to English. Applying these filters, we initially retrieved 945 papers. Our inclusion and exclusion criteria are summarized in Table 1.

Manual screen and snowballing. After the initial retrieval, we conducted a manual screening based on the title, abstract, and the keywords of each record to assess whether the study concerned generated code and discussed related to NFQCs. To ensure the quality of the included studies and our literature review, we further examined the studies along two predefined questions: Does the study address and examined quality of generated code beyond functional correctness? Does the study include empirical evaluation, systematic analysis, or concrete tooling? Only studies that satisfied both screening questions were retained. Following this manual screening process, a total of 107 papers were included in the dataset. In addition, during the manual screening phase, we applied forward and backward snowballing to the included studies to identify

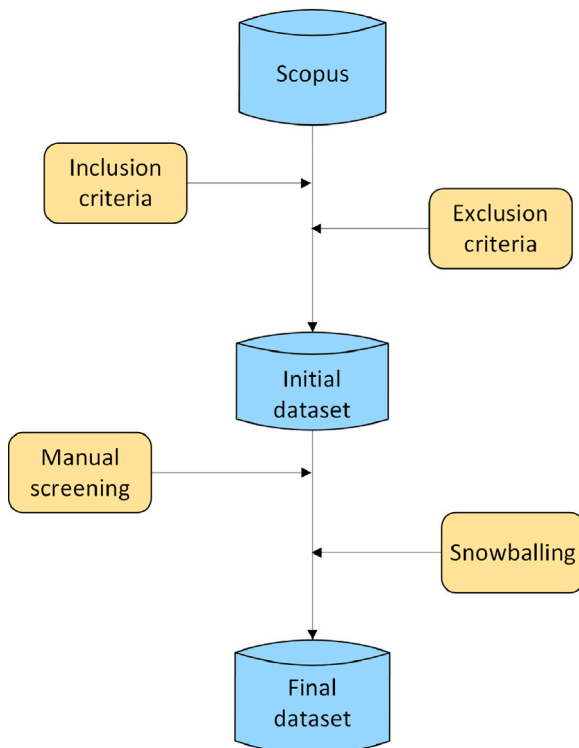


Fig. 1. Search and selection process.



Fig. 2. Wordcloud of the identified literature.

further relevant literature and found two more studies. Following this process, the final dataset comprises 109 papers. The structure of the applied search and selection process is shown in Fig. 1. To show an overall picture of the topics covered in the final dataset, we generated a wordcloud based on the keywords of the papers, as depicted in Fig. 2

Data extraction and analysis. Two authors were involved in the review and validation of the analysis. We created a spreadsheet to extract key information we wanted to analyze. In the spreadsheet, we extracted the primary information from the 109 selected papers, including title, publication year, authors, venue, evaluation metrics and tools, quality attributes addressed, LLMs evaluated, programming languages examined, and results.

During the data extraction and analysis process, two authors mapped the quality attributes discussed in the studies to the corresponding categories defined in the ISO/IEC 25010 quality model. Studies were allowed to contribute to multiple quality attribute categories when multiple attributes were examined. The mapping results were then cross-reviewed by the two authors. Any inconsistencies were discussed together until consensus was reached.

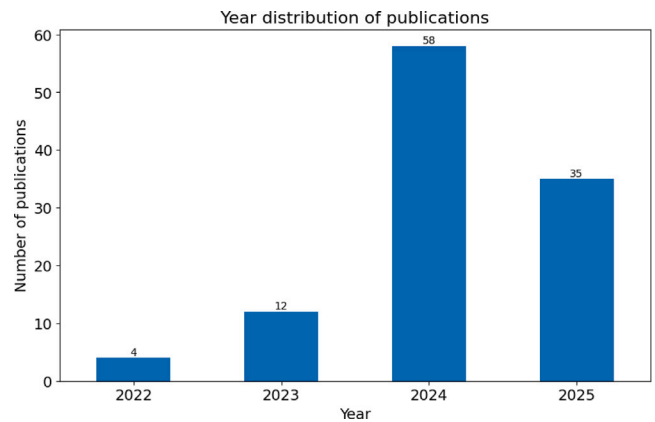


Fig. 3. The year distribution of papers.

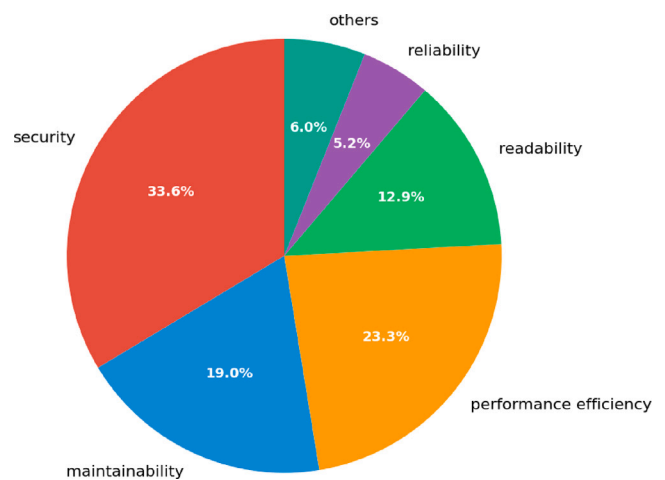


Fig. 4. The distribution of papers by NFQCs.

After completing data extraction and the initial analysis, we synthesized the collected evidence to address the research questions. For RQ 3.1, we analyzed publication years, quality attributes addressed, and programming languages examined across all included studies to identify temporal trends and shifts in research focus. For RQ 3.2, we examined the evaluation metrics and tools used, and LLMs evaluated, to characterize the current state of evaluation practices and to identify potential research gaps and opportunities.

3.2. Literature review findings

What trends can be observed? Following the search strategy described, 109 studies were finally included. The dataset includes 31 journal articles and 68 conference papers published across 62 distinct journals and conference venues. Table 2 summarizes the main publication venues and representative studies.

As shown in Fig. 3, the publication trend shows a rapid increase in studies on NFQCs of generated code between 2022 and 2024. While only four papers appeared in 2022 and 12 in 2023, the number surged to 58 in 2024. The decrease in 2025 does not necessarily indicate reduced research attention. Rather, it may reflect the fact that a number of recent studies still exist as preprints. This growth indicates that the research community has shifted from investigations of functionally correct code generation toward high-quality code generation, including several non-functional quality attributes.

Fig. 4 illustrates the distribution of NFQCs addressed by the included studies, based on our mapping to the ISO/IEC 25010 quality

Table 2
Primary publication venues.

Venue	Primary publication venue	Primary study
Journal	Journal of Systems and Software	Moradi Dakhel et al. (2023)
Journal	IEEE Transactions on Software Engineering	Khojah et al. (2025), Liu et al. (2024) and Zhao et al. (2025)
Journal	Empirical Software Engineering	Börstler et al. (2023), Z. Zheng et al. (2024) and Asare et al. (2023b)
Journal	ACM Transactions on Software Engineering and Methodology	Ouyang et al. (2024), Chen et al. (2025) and Fu et al. (2025a)
Conference	International Conference on Mining Software Repositories	Moratis et al. (2024), Nguyen and Nadi (2022b) and Siddiq et al. (2024b)
Conference	International Conference on Automated Software Engineering	Ren et al. (2024) and Siddiq et al. (2024a)
Conference	International Working Conference on Source Code Analysis and Manipulation	Siddiq et al. (2022)
Conference	International Conference on Evaluation and Assessment in Software Engineering	Coignon et al. (2024b) and Della Porta et al. (2025)
Conference	ACM SIGSAC Conference on Computer and Communications Security	Klemmer et al. (2024)
Conference	International Conference on Learning Representations	Shypula et al. (2024b)

Table 3
Thematic overview of NFQC evaluation in generated code.

NFQCs	What is evaluated	Typical methods	Representative studies
Security	Presence of security vulnerabilities, security smells	Static analysis and human review	Tony et al. (2023), Tony et al. (2025), Pearce et al. (2022) and Siddiq et al. (2022)
Maintainability	code smells, readability, modularity	Static analysis, human review	Shehab et al. (2024), Moratis et al. (2024), Liu et al. (2024a) and Le et al. (2024)
Performance efficiency	runtime behavior, memory usage	Execution-based benchmarking (LeetCode, gem5 simulator)	Shehab et al. (2024), Niu et al. (2024a), Sakib et al. (2024) and Huang et al. (2024a)
Readability	readability violations, code smells	PMD, static analysis tools, human review	Moratis et al. (2024) and Ságodi et al. (2024)
Reliability	misuse of APIs	ROBUSTAPI benchmark	Zhong and Wang (2024)

model. Security emerges as the most frequently examined attribute, accounting for 33.6% of the studies. Performance efficiency (23.3%) and maintainability (17.2%) also receive substantial attention. In comparison, readability represents 12.9% of the studies, while reliability (5.2%) and modularity (1.7%) are addressed far less often. The remaining 6.0% of studies fall into an “others” category, covering quality aspects that are either less commonly investigated or do not map cleanly to a single ISO/IEC 25010 attribute. The distribution suggests that existing research tends to prioritize certain quality attributes over others, suggesting a potential research gap for future investigation.

In the results, we examined the distribution of programming languages evaluated across different NFQCs. Overall, Python is the most frequently studied language across all NFQCs, particularly in evaluations of security (44.1%), performance efficiency (60.9%), and maintainability (66.7%). Other programming languages are less commonly examined in isolation and are more often included alongside Python within multi-language evaluation settings. These patterns suggest that current evaluation practices are influenced by language availability and execution convenience, with Python-centric benchmarks shaping much of the existing evidence on LLM code generation.

How NFQCs are examined and the limitations. To answer RQ 3.2, we summarized representative studies on the evaluation of NFQCs in generated code, together with the typical evaluation methods they employ and the metrics used to reflect NFQCs, as shown in Table 3.

Taken together, existing studies provide a partial but structured view of how NFQCs of generated code are currently evaluated. The following paragraphs summarize the state of research across three main dimensions.

- **Security** A growing body of studies has observed that LLM-generated code can be functionally correct while containing security vulnerabilities (Perry et al., 2023; Khoury et al., 2023; Pearce

et al., 2022; Elgedawy et al., 2024; Siddiq and Santos, 2022; Asare et al., 2023b). Most existing work evaluates security using static code analysis tools, such as CodeQL, Bandit, or SonarQube, which detect security issues using the presence of predefined vulnerability patterns or insecure coding practices (Pearce et al., 2022; Liu et al., 2024). Besides the widespread use of static analysis tools, several datasets and benchmarks have been introduced to support systematic security evaluation, including SecurityEval (Siddiq and Santos, 2022), LLMSevEval (Tony et al., 2023) and AI Code Generators for security (Natella et al., 2024). These benchmarks improve reproducibility across studies, but they primarily focus on isolated code snippets and a limited set of vulnerability types. As a result, security evaluation in the current literature remains largely constrained to what can be detected through static analysis at the code level. A small set of studies compares the security of generated code with human-written code using manual inspection (Asare et al., 2023b). These comparisons suggest that the generated code is not uniformly worse than human-written code in terms of introducing vulnerabilities, but differences are observed in the types and severity of issues detected. Overall, existing studies consistently indicate security concerns in generated code, while also reflecting a strong methodological reliance on static, code-level evaluation approaches.

- **Performance Efficiency** Performance efficiency of generated code is mainly evaluated using runtime performance and memory usage (Niu et al., 2024a; Singhal et al., 2024; Coignon et al., 2024b; Huang et al., 2024b; Waghjale et al., 2024; Shypula et al., 2024b). The studies measure these properties by executing generated solutions on standardized programming tasks, often drawn from platforms such as LeetCode. Execution time and memory consumption are then compared with human-written

or reference solutions. Under this setup, many studies report that functionally correct generated code achieves performance comparable to human-written solutions. In some cases, generated solutions even outperform human-written code (Coignon et al., 2024b). A small number of studies also consider energy consumption (Vartziotis et al., 2024; Solovyeva et al., 2025). These studies typically measure power usage during program execution and analyze it together with runtime and memory usage. The results suggest that LLMs do not optimize for energy efficiency by default. Energy improvements are usually observed only when energy consumption is explicitly included in the evaluation or optimization process.

Overall, current performance evaluations rely on a limited set of measurable indicators, mainly runtime and memory usage. These evaluations are often conducted on isolated problems. This approach supports controlled and reproducible experiments, but it also restricts performance analysis to settings where execution behavior is easy to benchmark. Interactions between performance efficiency and other quality characteristics are sometimes reported, but they are rarely examined in a systematic manner.

- **Maintainability** Maintainability refers to how easily code can be understood and modified by developers over time. In the literature, maintainability of generated code is most commonly evaluated using static analysis tools and predefined metrics. Tools such as SonarQube, CodeQL and PMD are frequently used to identify code smells, complexity issues, and structural problems in generated code (Nguyen and Nadi, 2022b; J. Zheng et al., 2024; Liu et al., 2024a; Yetistiren et al., 2023). Some studies show the reports from static analysis, which report low complexity and reasonable structure in generated code (Nguyen and Nadi, 2022b). However, other studies combined manual review and static analysis tools (Liu et al., 2024a); they report that such code can still be verbose or poorly organized, which makes it harder to maintain in the future. This suggests that static metrics reflect only part of maintainability and may overlook issues that affect long-term maintenance.

Readability is not an attribute defined ISO/IEC 25010. Existing studies evaluate readability using benchmarks or proxy metrics derived from static analysis (Weyssow et al., 2025). Their results suggest that code generated by GPT-4 is more readable than that generated by open-source models, likely due to the well-formatted and structured training data. Besides readability, some studies also investigate the modularity and structure of the generated code. The study also reported that LLMs often generate code with poor modularity and structure, making it difficult for developers to understand and modify. Kang et al. (2024) questioned the traditional assumption that modularity improves code quality for LLM-generated code, finding that modular code does not consistently enhance performance and may not be favored by LLMs during generation.

Several studies compare maintainability between LLM-generated and human-written code using the same analysis tools (Licorish et al., 2025; Eltabakh et al., 2024). These comparisons yield inconsistent results. In some cases, human-written code is found to be more readable and better aligned with coding standards. In other cases, generated code appears more structured and better documented, particularly for simpler tasks. Overall, these results suggest that maintainability outcomes depend strongly on task complexity, prompt design, and evaluation method.

Taken together, current results indicate that LLMs can produce readable and partially maintainable code, but maintainability is not ensured by default. Most evaluations rely on static, code-level metrics, and human intervention remains necessary to address structural and long-term maintenance issues.

3.3. Discussion of literature review findings

Our analysis of the literature reveals that the research community has made some progress in evaluating the quality of generated code. However, several challenges and open gaps need to be addressed before NFQC evaluation in this context can become systematic and comparable across studies.

- **Ambiguity and inconsistency in defining quality characteristics.** Existing studies often use overlapping or loosely defined terms to describe quality characteristics, such as using readability, maintainability, or code quality interchangeably. While ISO/IEC 25010 provides a well-established taxonomy for software quality, it has not been systematically adapted to the context of generated code. As a result, studies redefine quality characteristics based on available tools or metrics, rather than on a shared conceptual framework. This lack of alignment complicates the interpretation of results and makes it difficult to determine whether different studies are evaluating comparable aspects of quality, even when similar terminology is used.
- **Imbalance in evaluated quality characteristics.** The literature mainly focuses on a small number of NFQCs, most notably performance efficiency, security, and maintainability. These characteristics are clearly important in practice, but their frequent appearance in research is also linked to the fact that they can be evaluated using existing automated, code-level tools. Other quality characteristics defined in ISO/IEC 25010 receive much less attention. This is not necessarily because they are less important, but because their evaluation depends on system context, runtime behavior, or user interaction, which are difficult to assess from isolated code artifacts. As a result, current NFQC coverage is influenced by what can be easily measured, and this may lead to a skewed view of the overall quality of generated code.
- **Lack of a unified quantitative evaluation framework.** Researchers measure the functional correctness of the generated code using accuracy or $pass@k$, which measures how often LLMs generate a correct response on its k_{th} attempt for a given problem. In contrast, NFQC evaluation relies on a heterogeneous set of metrics, benchmarks, and tools, often tailored to individual studies. This methodological diversity limits comparability and reproducibility, as differences in reported results may stem from evaluation choices rather than underlying model behavior. More importantly, the absence of a common framework prevents joint analysis of functional correctness and multiple NFQCs, reinforcing a fragmented view of code quality that treats each characteristic in isolation.
- **Limited understanding of interactions and trade-offs among NFQCs.** Most studies assess NFQCs only for code that is already functionally correct, and some report trade-offs between correctness and individual quality characteristics. However, interactions among multiple NFQCs are rarely examined in a systematic way. Unlike traditional software engineering, where trade-offs are largely determined by design decisions and constraints, LLM-based code generation introduces new degrees of freedom through prompting strategies, model selection, and generation workflows. These mechanisms suggest that trade-offs may be influenced or steered rather than simply observed. The lack of empirical studies that explicitly investigate such controllability limits our understanding of how NFQCs jointly evolve in generated code.

In summary, current research on NFQCs in generated code is inconsistent in terminology, selective in scope, and inconsistent in evaluation methods. Addressing these gaps will require unified definitions, standardized benchmarks, and comprehensive evaluation frameworks to improve understanding of the quality of generated code.

4. Workshops

To complement and validate the findings of our literature review, we held two interactive workshops with industry experts from several organizations. These organizations are actively exploring the integration of LLMs into large-scale software systems and seeking to ensure the reliability of LLMs' outputs. The workshops are designed to answer the following research questions:

- **RQ 4.1:** What NFQCs are considered most important by industry practitioners when evaluating generated code?
- **RQ 4.2:** What challenges and risks do practitioners perceive when integrating generated code into existing development workflows?

4.1. Workshop design

Participants. We invited several groups of industry experts to participate in the workshops, with a total of 15 participants, including three of the authors. The participants are from seven different organizations in various industry areas, and most of the organizations are members of Software Center³ (SWC), which is an industry-academic collaborative network to accelerate industrial digitization and the adoption of novel approaches to software engineering. The participants have a wide range of professional skills, including quality-in-design, system integration and testing, CI/CD architecture, and automotive software test-driven development. Their professional experience in software engineering ranges from 3 to over 13 years, ensuring both breadth and depth of knowledge.

Workshop procedure. The workshops were held remotely through Microsoft Teams and divided into two sessions to accommodate different groups and foster focused discussion. In the first session, we met experts from a non-SWC organization, and later in the second session, we met SWC organization participants. During the workshop, we intentionally made the discussion open-ended, and questions were welcomed at any time to encourage spontaneous and candid input. Both sessions followed the same overall structure:

- **Presentation of our findings:** Each session began with a presentation of our literature review, highlighting the recent trends and the key findings related to NFQCs in LLM-generated code. The presentation acted as a shared foundation for discussion, providing participants with a common frame of reference for the subsequent workshop activities.
- **Participant sharing:** After our presentation, the participants were invited to present their ongoing projects, observed challenges, and useful practices related to this topic. This segment enabled different organizations (in the second workshop) to communicate with each other on the practical implementations and establish the synchronization between industry and academia.
- **Open discussion:** The discussion phase happened not only during the presentations, but also after the presentations. We did not follow a predetermined set of questions. Instead, it was facilitated in an exploratory manner, allowing participants to raise issues most relevant to their contexts and to reflect on both the presentation and their practical experience.

Data collection and analysis. During the two sessions, two of the authors independently took notes during each session. After each workshop, the presentation slides were compiled, and all authors had meetings to consolidate the individual notes into a comprehensive workshop transcript. In cases where discrepancies arose between the two notes, these were discussed and resolved through consensus.

For data analysis, we used the definitions from ISO/IEC 25010 to systematically examine and categorize the workshop results. Unclear cases were discussed until the authors reached an agreement. Finally, the categorized data were synthesized to identify patterns and to highlight which aspects of NFQCs were emphasized during the workshops.

4.2. Experience of practitioners

In this section, we summarize the presentations and the discussions during the workshops. For anonymity, the companies are labeled from C1 to C7.

In the first workshop, C1 reported that in their organization, the adoption of LLM-based development tools is still at an early stage, and issues encountered during use are reported to support further model and tool improvement. Security risks in generated code were repeatedly raised, particularly for C and C++. Two QA arrangements were discussed: applying existing QA processes used for human-written code, which was seen as ensuring quality but reducing the potential efficiency improvement for using AI, and a hybrid human-AI setup, which was considered more efficient but potentially riskier if oversight is insufficient. Across discussions, maintainability emerged as the dominant concern for generated code, especially for large codebases with legacy components aged 10 to 15 years. Participants also noted that current LLMs lack sufficient contextual understanding of large codebases, limiting their effectiveness in such settings, while safety was regarded as important for them, but typically addressed at the system level rather than the code level.

In the cross-company workshop, C2, C3, and C4 presented their ongoing work relevant to NFQCs of generated code, and others joined the discussions. C2 presented several ongoing and completed projects related to AI-assisted software development, including *prompt design for generating readable and clean code*, *generative AI for UX design*, *AI-based test generation and code review*. During the discussion, particular interest was expressed in the work on prompt design for generating readable and clean code, while participants also noted the persistent difficulties in measuring maintainability and raised questions about extending evaluations to larger code units.

C3 presented exploratory studies evaluating the quality of generated code using different toolchains. They shifted the focus from functional correctness toward NFQCs, especially maintainability and readability. Participants observed that small code fragments generally yield acceptable results, while larger and more complex systems exhibit poor readability and maintainability, and in some cases reproduce known vulnerabilities. They shared their mitigation strategies on this issue, which included decomposing tasks into smaller steps and adopting agent-based workflows using multiple models. The key challenges discussed were scalability, availability of high-quality training data, and prompt management. They also suggested using systematic code review, test-driven development, and learning from feedback to ensure the quality of generated code.

C4 presented work on combining formal methods with LLM-based code generation. Their approach integrates formal specifications with informal documentation as model input to achieve reliable and correct code generation. In their practices, they observed that zero-shot prompting produced unexpectedly good results. In the discussion, participants suggested that expectations for generated code may exceed those typically applied to human-written code.

During the open discussion, all participants engaged in in-depth discussions based on the work presented in the workshop. Several open questions were proposed and discussed. The discussions were summarized in Table 4.

³ <https://www.software-center.se/>

Table 4

Summary of key observations and open questions from the workshop discussion.

Topic	Observation	Notes
Functional correctness	Functional correctness was emphasized as a prerequisite for generated code.	Treated as a baseline requirement rather than a trade-off dimension.
Applicability of ISO/IEC 25010 NFQCs	Not all ISO/IEC 25010 quality characteristics were considered applicable at the code level. Safety and compatibility were described as system-level properties, while maintainability and security were viewed as assessable at the code level.	Some attributes cannot be evaluated at the code level.
NFQC prioritization	Performance efficiency and security were acknowledged as important, but maintainability was consistently emphasized as the primary concern in industrial settings.	Especially relevant for large and long-term codebases.
Maintainability metrics	Participant expressed dissatisfaction with existing maintainability metrics and questioned their ability to reflect practical maintenance effort.	Reported as a limitation of current evaluation approaches.
Readability	Readability was frequently raised as an important quality attribute of generated code and was commonly associated with maintainability.	Participants expressed interest in tool support for assessing readability, particularly in large projects.
Trade-offs among NFQCs	Interactions among different NFQCs were perceived as underexplored compared to trade-offs between correctness and individual NFQCs.	Open questions raised regarding whether such trade-offs are inherent and how specific NFQCs might be prioritized or tuned during development.

4.3. Discussion of workshop findings

- **Reconsidering the scope of NFQC Evaluation.** The difference between code-level and system-level NFQCs has clear implications for how generated code is currently evaluated. The discussion in the workshop suggests that the frequent focus on maintainability, security, and performance efficiency in the literature is not only due to their importance, but also because these attributes are easier to measure at the code level using existing tools such as static analysis and linters. This emphasis on what is measurable risks narrowing the scope of NFQC research, as it may blur the line between what can be evaluated and what should be evaluated. In contrast, NFQCs such as usability, compatibility, and reliability depend on system context or user interaction. As a result, they are difficult to assess from isolated code snippets and are less frequently studied. Workshop discussions suggest that this focus on code-level measurability may introduce a systematic bias in current evaluations of generated code. This highlights the need for future research to more clearly justify the chosen evaluation level and to consider complementary approaches that go beyond code-level metrics.

- **Prioritized quality characteristics for generated code.** The emphasis on maintainability and readability from the practitioners highlights their concerns about the long-term sustainability of software systems that incorporate generated code. While generated code may be functionally correct, workshop discussions suggest that its increasing volume and complexity can make code comprehension more demanding, particularly when developers rely heavily on LLMs during development. This shift introduces a tension between short-term productivity gains and long-term maintenance effort. Importantly, the lack of reliable and widely accepted metrics and tools to assess the maintainability and readability in practice becomes an issue. This gap suggests that current research and tooling may be insufficient to support practitioners' needs in practice. As LLMs are increasingly used to generate larger and more complex code structures, the challenge may move from identifying defects to understanding and evolving the generated code, further underscoring the need for research that better assesses maintainability and readability in the context of generated code.

- **The impact of trade-offs and enhancing specific quality characteristics.** A significant point of discussion centered on the complex nature and implications of trade-offs among the various quality attributes within generated code. While existing research has examined trade-offs between individual NFQCs and functional correctness (Waghjale et al., 2024; Coignon et al., 2024b), implicitly assuming that other quality attributes can be considered independently or held constant. In contrast, practitioners emphasized that in real development settings, multiple NFQCs interact simultaneously, and improvements in one dimension frequently influence others in a non-linear way. This suggests that trade-offs may not be inherent properties of the code alone, but emergent outcomes shaped by development workflows, tooling choices, and usage context.

Additionally, workshop participants did not frame trade-offs solely as constraints to be observed, but as properties that developers actively attempt to manage. Rather than asking whether NFQCs from a zero-sum relationship, discussions focused on how specific quality attributes can be prioritized under different constraints, such as emphasizing security in security-critical systems. This shift highlights a gap between current empirical evaluations, which largely report trade-offs post hoc, and industrial needs for mechanisms that enable the deliberate control and tuning of NFQCs during code generation. Addressing this gap requires moving beyond static evaluations toward experimental design and workflows that explicitly account for multi-objective optimization and quality steering in LLM-assisted development.

Overall, the workshop discussions reveal three main insights. First, only a subset of NFQCs can be automatically and reliably measured at the code level, as there are established tools such as static analysis and linters. Second, practitioners emphasized that maintainability and readability are important aspects given their direct impact on long-term projects and complex systems. Third, participants highlighted the need to better understand and manage trade-offs between functional correctness and individual NFQCs. Future work should explore how multiple NFQCs interact and how to manage these trade-offs effectively. Together, these insights outline the current landscape of NFQC evaluation of generated code and motivate the empirical study presented in the following section.

Addressing the questions raised during the workshop regarding these NFQC trade-offs and interactions, the subsequent section of this paper details the establishment of our dataset and methodology specifically designed to investigate the interactions among NFQCs in generated code.

5. Trade-offs among NFQCs in generated code

5.1. Motivation and research questions

According to the findings from our literature review and workshops, we identified three NFQCs that consistently emerge as primary concerns in both academia and industry: maintainability, security, and performance efficiency. Thus, we deliberately scoped our experiment to these three dimensions. For performance efficiency, we further considered two measurable dimensions widely reported in the literature: runtime and memory usage.

Despite the prominence of these NFQCs, both our literature analysis and workshop discussions suggest that current studies largely examine these characteristics in isolation, with limited investigation into the interaction patterns and potential trade-offs among NFQCs of generated code in real-world development scenarios. This practical concern motivated us to study NFQCs with realistic software engineering tasks that reflect how generated code is actually used and evaluated. In this empirical study, we focus on analyzing the behavior and interactions of NFQCs in real-world, repository-level scenarios. This setting allows us to examine NFQCs as they arise in practice and to better understand their interaction patterns under realistic development conditions.

- **RQ 5.1:** How do LLMs perform on the selected NFQCs when generating code for real-world software engineering tasks?
- **RQ 5.2:** Can incorporating static analysis feedback, such as results from CodeQL, into prompts improve the corresponding NFQC performance of generated code?
- **RQ 5.3:** Does improving one NFQC result in trade-offs with other NFQCs?

5.2. Experiment design

To address these research questions, we designed a three-stage experimental pipeline, as summarized in Fig. 5. In the **first stage (Baseline Evaluation)**, we generated patches for a benchmark dataset of real-world software issues (Jimenez et al., 2024) and evaluated both the generated and the benchmark gold patches. This stage established baseline measurements of functional correctness, runtime, memory usage, and static analysis results related to security and maintainability. In the **second stage (Filter and Prompt Design)**, we identified a set of comparable, correctly resolved instances across models and used the corresponding baseline results to design targeted prompts for NFQC-specific optimization. In the **final stage (NFQC-specific Regeneration and Evaluation)**, we regenerated the patches using the new prompts, evaluated them under the same conditions, and compared the results against the baseline to analyze improvements and trade-offs among different NFQCs.

Dataset selection. The goal of the experiment is to evaluate the NFQCs of generated code and explore how different NFQCs interact in functionally correct, repository-level code changes produced by LLMs. To this end, we selected *SWE-bench Lite* (Jimenez et al., 2024), which is widely used by the community, as the evaluation benchmark for our experiment. Each task in *SWE-bench Lite* is derived from a real GitHub issue and requires generating a patch that passes the corresponding test suite. The resulting code changes typically span multiple files and interact with existing project dependencies, which makes this benchmark particularly suitable for assessing NFQCs at the repository level. In our experiments, we used the *test* set of *SWE-bench Lite*, comprising 300 issue-pull request pairs from 11 Python repositories, predominantly drawn from Django (127 instances) and SymPy (80 instances). Each instance is evaluated by executing unit tests, using the post-pull request behavior as the reference.

In addition, *SWE-bench* provides an established evaluation infrastructure, including the *SWE-agent* framework, which supports reproducible and automated patch generation and verification. This allows

us to conduct our experiments within a standardized setup while focusing our analysis on the NFQCs of generated patches. Fig. 6 shows the structure of each instance in *SWE-bench Lite*.

Model selection. While LLMs deliver promising performance in generating basic functions and algorithms, their performance on the *SWE-bench* family benchmarks remains limited. As of August 20, 2025, Claude 4 Opus achieves a resolved rate of 67.60% on *SWE-bench*, and Claude 4 Sonnet reports 66.93% (submission on May 21, 2025) and 56.67% (submission on May 26, 2025) on *SWE-bench* and *SWE-bench Lite*, respectively.⁴

For model selection, we reviewed all the submissions on the *SWE-bench Lite* leaderboard and aimed to minimize confounding factors unrelated to the models themselves. To this end, we decided to use the *SWE-agent* framework as the generation pipeline. *SWE-agent* is an agent-based system designed to automate the generation process in LLM code generation. It enables LLMs to use tools, interact with the environment, and produce patches step by step, without human instruction. The use of *SWE-agent* also ensures that the generation process is fully driven by the LLM, thereby reducing potential biases introduced by manual intervention. This design makes comparisons across models more fair and enhances the reliability of the evaluation. To examine how models differ in their NFQCs improvement potential, we selected three models that combine representativeness and contrastiveness:

- **Claude 4 Sonnet:** A mid-size model in the Claude 4 family. It demonstrates strong coding and reasoning abilities with enhanced instruction following and extended context support.
- **DeepSeek-Reasoner:** A reasoning-augmented model that generates chain-of-thoughts to improve the generation accuracy. It shows competitive performance on reasoning and code-related tasks compared to state-of-the-art closed-source models.
- **GPT-4o:** OpenAI's LLM, released in 2024. It achieves performance on par with GPT-4 Turbo in code generation, while offering faster inference and lower API costs.

These models cover closed-source and open-source models, providing a more representative and generalizable comparison for evaluating NFQC in generated code.

Generation strategy. In our experiment, we employed *SWE-agent* (J. Yang et al., 2024) as the generation framework. Each model has its own identical configuration parameters, and interacts with the environment through command-line tool. The agent supports various stopping criteria and cost control strategies. In the first stage, we set a cost limit of \$1 per instance for each model, without restricting the number of retries. This configuration balanced computational cost with the ability to maximize resolved instances, which provides more baseline resolved patches for subsequent analysis. In the final stage, we limited the number of retries to 1, where the agent produced one patch per instance before moving to the next instance. This multi-round generation design more closely mirrors how humans interact with LLMs in practice (Zhong et al., 2024). The configuration is shown in Fig. 7.

Evaluation system. The evaluation system was designed to assess the generated patches across three NFQC dimensions: **performance efficiency** (including runtime and memory usage), **security**, and **maintainability**, along with functional correctness as a prerequisite. Thus, we measured four metrics in the experiment. It consisted of two main components: The first component focused on functional verification and performance measurement. For each instance, we used Docker to create an isolated environment, cloned the corresponding repository, applied the generated patch, and executed the test cases provided by *SWE-bench Lite*. This step verified whether the patch was functionally correct and resolved the given issue. For patches that resolved the given

⁴ <https://github.com/SWE-bench/experiments>

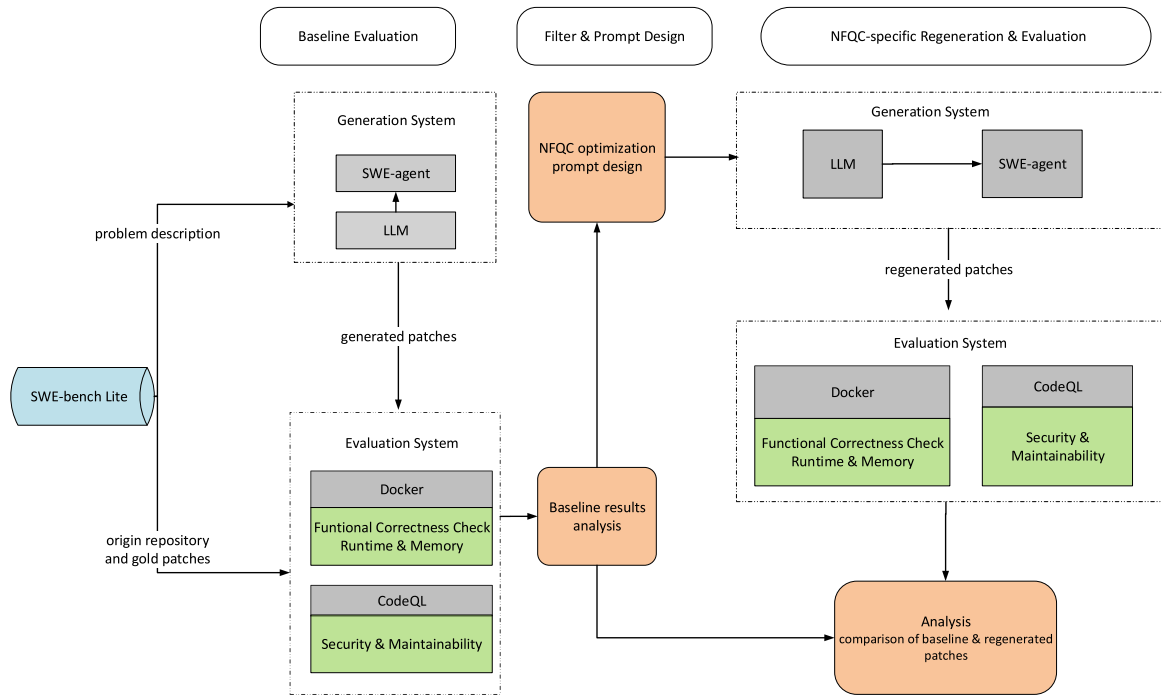


Fig. 5. Overview of the experiment procedure. The workflow is divided into three main stages: (1) **Baseline Evaluation**, where initial patches were generated using SWE-agent and different LLMs, generated and benchmark gold patches are evaluated with Docker and CodeQL; (2) **Filter and Prompt Design**, where baseline results are analyzed and NFQC-specific prompts are constructed; and (3) **NFQC-specific Regeneration and Evaluation**, where patches are regenerated using NFQC-specific prompts, re-evaluated and compared against the baseline to analyze improvements and potential trade-offs.

SWE-bench Lite Instance Structure

instance_id: Unique identifier of the instance.
 repo: GitHub repository name.
 base_commit: Commit hash before applying the fix.
 problem_statement: Title and description of the issue.
 hints_text: Optional hints or comments from the issue.
 test_patch: Unit test modifications introduced by the PR.
 patch: Reference solution patch.
 FAIL_TO_PASS: Tests that failed before and passed after.
 PASS_TO_PASS: Tests that passed both before and after.

Fig. 6. SWE-bench Lite Instance Structure.

SWE-agent Configurations

```

per instance cost limit: 1
per instance call limit: 0 (means no limit)
temperature: 0.0
top_p: 1.0
parse_function type: function calling (for Deepseek, set to thought
action)
  
```

Fig. 7. SWE-agent Configurations.

issues, we further measured the total execution time required to run the entire test suite and the peak memory usage during the execution.

The second component performed static code analysis to evaluate security and maintainability. We selected CodeQL due to its widespread industrial usage and ability to yield reproducible results. Specifically, we performed holistic repository analysis rather than incremental scans to ensure comprehensive detection. We applied the official Python security and quality query suites released by GitHub for static analysis of source code. Following the official CodeQL documentation, we filtered these suites based on their metadata tags (e.g., *maintainability*, *security*). This screening resulted in a specialized evaluation set comprising 39 maintainability queries and 46 security queries. Table 5 lists the most frequently triggered maintainability rules, providing a granular view of the technical debt and maintainability risks identified during our experiments. We report the number of issues triggered by the patches, categorized by CodeQL's severity levels: *Error* (critical flaws), *Warning* (problematic patterns), and *Recommendation* (suggested improvements). Additionally, to mitigate potential tool-bias and strengthen our findings, we also reported **patch size (Lines of Code, LOC)**.

Experimental setup. Our experiment was conducted on a laptop with an AMD Ryzen 7 CPU and 16 GB RAM. The operating system was Ubuntu 22.04 with Python 3.11. We used SWE-agent v1.1.0, the latest release available as of May 2025, along with SWE-bench.

The details of the three stages are as follows:

- **Baseline evaluation:** We first established baseline measurements for the original repositories and SWE-bench Lite gold patches across the selected NFQCs using the evaluation system described earlier. These measurements established the baseline results for subsequent comparison. After that, we generated the initial patches using the SWE-bench Lite and the SWE-agent. SWE-agent functioned as a unified framework that coordinated the patch generation process across different LLMs. Since the official SWE-bench Lite leaderboard already provides submissions using SWE-agent with Claude 4 Sonnet and GPT-4o, we reused the same configurations to reproduce the results. For DeepSeek-Reasoner, although no official submission was available, we used the same configuration to maintain fairness across models. Thus, in this stage, the configurations were identical across experiments, with the only difference being the LLM used. After patch generation, the SWE-agent output the generated patches corresponding to the instances in the dataset. These patches were then evaluated through the evaluation system, performing the functional correctness check and the NFQC evaluation. An instance was considered *resolved* if its generated patch successfully fixed the issue described in the prompt. For all resolved instances, we additionally measured the runtime and peak memory usage during the functional correctness check. The output from this stage included both the generated patches and the evaluation results of the original repositories with and without the generated patches applied.
- **Filter and prompt design:** Next, we identified the resolved instances for each model and derived their intersection, meaning we retained only those instances that were successfully resolved by all three models. This ensured that subsequent analyses compared the same issues under identical functional conditions. For these common instances, we analyzed the corresponding NFQC evaluation results from CodeQL and recorded the number and type of newly introduced maintainability and security issues. Using these results, we designed NFQC optimization prompts by adapting prompts from prior studies and SWE-bench to approximate how developers might provide targeted feedback to an LLM in practice (J. Yang et al., 2024). Building on the prompts used for the initial generation, the optimized prompts introduced

two additional blocks: *prev_patch_block* and *feedback_block*. The *prev_patch_block* provided the correct patch generated in the first round, while the *feedback_block* gave the feedback on the specific quality dimension. For security and maintainability, the feedback block contained information derived from CodeQL reports to help the model locate the issue, such as the triggered ruleids, a short description of the rule, and their corresponding locations, together with the targeted NFQC to be improved. For runtime and memory optimization, the feedback block only specified the dimension to be improved, without providing more detailed guidance. The structure of the NFQC optimization prompts is shown in Listing 1.

Listing 1: NFQC-specific optimization prompt

```

1 PROMPT_TMPL_BASE = """Your previous patch is
  functionally correct. You must re-
  implement its behavior directly on the base
  commit, while focusing on improving **{
  focus}**. Always generate a unified diff
  against the ORIGINAL BASE COMMIT.

2
3 <issue>
4 {problem_statement}
5 </issue>
6
7 {prev_patch_block}
8 {feedback_block}
9 REQUIREMENTS:
10 - Output ONLY a unified diff** (git patch)
   with file paths relative to the repo root.
11 - Do NOT include explanations, markdown fences,
   numbering, or shell commands.
12 - Keep changes minimal; do NOT modify tests.
13 {requirement_line}
14 - If a trade-off arises, prioritize functional
   correctness.
15 - Re-implement the behavior of the previous
   patch on the base commit, preserving
   functionality.
16
17 FORMAT:
18 - Start with lines like: diff --git a/<path> b/<
   path>
19 - Use standard unified diff hunks: @@ -old,+new
20 @@- Ensure the patch applies cleanly.
21
22 Now produce ONLY the patch:
23 """

```

- **NFQC-specific regeneration and evaluation:** In this stage, for each model and each instance in the intersection set, we applied the designed optimization prompts separately, producing four regenerated patches per instance per model. Every regenerated patch was evaluated with the same pipeline as in the first stage: functional correctness in isolation, followed by measurement of total runtime and peak memory if resolved, and a holistic CodeQL scan to examine security and maintainability. We then compared these outcomes against the corresponding unoptimized results for the same instance and model to quantify the improvements and assess trade-offs across NFQCs. (As in the baseline, only patches that resolve the original issue were included in comparisons.)

5.3. Experimental results

This section presents the results of our empirical study, following the research questions and the stages of our experimental design.

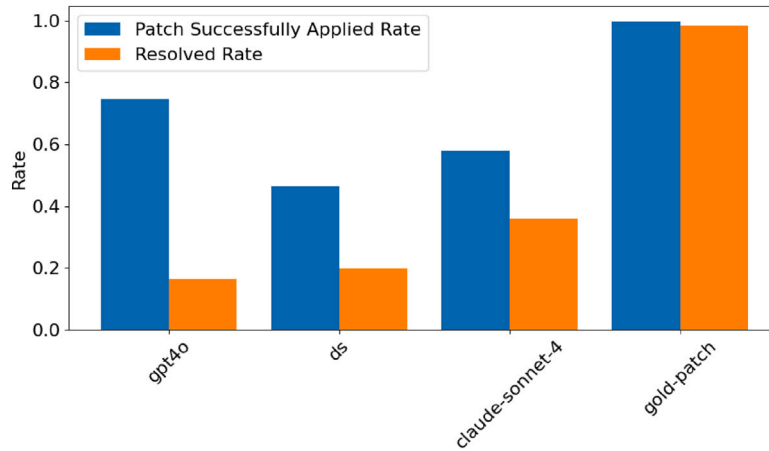


Fig. 8. Patch generation results of different models in the baseline evaluation. The total number is 300.

Functional correctness of the generated patches. Fig. 8 presents the comparative performance of the three models and the gold patches across 300 input instances. The result shows clear differences among the three models across the two metrics: the **patch successfully applied rate (PSA)** and the **resolved rate**. The PSA indicates the proportion of generated patches that could be merged without errors, regardless of whether it compiles or passes the tests, and the resolved rate measures the proportion of patches that successfully resolved the issues described in the prompts. In all cases, their performance falls well below that of the gold patches, underscoring a gap in functional correctness between LLM-generated patches and the curated reference patches produced by humans. GPT-4o has the highest PSA rate (75%) among the three models, but its resolved rate (16%) is the lowest. Claude-Sonnet-4 achieves a PSA rate of 58% together with the highest resolved rate of 36% among the three models. DeepSeek shows a more moderate performance, with a PSA rate of 46% and a resolved rate of 20%.

Among the failed attempts, we further inspected the agent logs to understand common failure causes. Notably, we observed that the agent's retries were largely devoted to resolving environment and shell command syntax errors (e.g., import failures) instead of iteratively refining the patch logic, suggesting a weakness in context and tooling rather than code generation itself.

Fig. 9 reports functional correctness for patches regenerated with prompts tailored to specific NFQCs. Besides PSA and the resolved rate, we included the submitted rate, which denotes whether the model provides a prediction for an instance. Under these NFQC-specific prompts, the functional correctness of the previously resolved patches decreased. The security prompt induces the biggest degradation in correctness, while memory and time have a lower impact on functional correctness. For GPT-4o, the submitted rate drops from 100% to 94% under the maintainability prompt, indicating that the model failed to generate patches for some instances. Under the security prompt, all three models show reduced submitted rates, with GPT-4o having the lowest resolved rate (12%). Claude-Sonnet-4 achieves the highest PSA and resolved rates under the maintainability, security, and time prompts, while DeepSeek-Reasoner achieves the best under the memory prompt, with both the PSA and the resolved rate reaching 94%.

RQ 5.1: Baseline performance across the selected NFQCs. To address RQ 5.1, we first evaluated the resolved generated patches from the first stage in terms of maintainability, security, runtime, and memory usage. To ensure comparability, the evaluation was conducted on the common subset of 17 resolved patches across the three models. Table 6 summarizes the CodeQL results on this common subset for maintainability and security, together with the average number of lines of code added by each patch. Following the official document of CodeQL, the rules are grouped by the severity level: Error, Warning and Recommendation.

Table 5

Distribution of maintainability issues across rules in gold and unoptimized generated patches.

RuleID	Severity	Gold-patch	GPT-4o	DeepSeek	Claude
cyclic-import	R	0	360	365	360
unused-global-variable	R	30	35	36	35
ineffectual-statement	R	0	8	8	8
unused-import	R	0	5	5	5
unused-local-variable	R	0	5	5	5

Overall, the generated patches trigger substantially more CodeQL rules than the gold patches, with a substantially larger number of newly added issues reported for maintainability than for security. In terms of patch size, the gold patches introduce an average of 2.65 added lines of code, whereas the generated patches vary across models, ranging from 2.35 lines for DeepSeek, 5.94 for Claude-Sonnet-4, to 16.71 lines for GPT-4o.

For maintainability, the gold patches introduce 30 recommendations and no warnings or errors. In contrast, the generated patches trigger more than 400 maintainability recommendations per model (423 for GPT-4o, 420 for DeepSeek, and 414 for Claude-Sonnet-4), together with two warnings in each case. No maintainability errors are reported for either gold or generated patches.

Additionally, the observed maintainability issues are not evenly distributed across rules. As shown in Table 5, the *cyclic-import* rule alone accounts for 360 to 365 newly introduced issues in the patches generated by LLMs, while no such issues are observed in the gold patches. In contrast, the *unused-global-variable* rule is introduced in comparable numbers by both gold and generated patches, with 30 occurrences in the gold patches and 35–36 occurrences across the generated patches. Other frequently triggered rules, such as *ineffectual-statement* and *unused-import*, occur only a small number of times and are absent from the gold patches.

For security, the overall number of triggered rules remains low across all models. As reported in Table 6, the gold patches trigger two security findings at the error level, while the generated patches trigger between four and six error-level findings. No security warnings or recommendations are reported for any patch type. All newly introduced security issues observed in the generated patches are attributable to the *clear-text-storage-sensitive-data* rule.

Table 7 reports the performance of the unoptimized patches with respect to test runtime and memory usage. All statistics are computed over the common subset of 17 resolved patches. The generated patches from three models exhibit higher average test runtime and memory usage. GPT-4o records an average runtime of 23.37 s and a memory usage of 44.54 MB, both of which are clearly higher than the gold

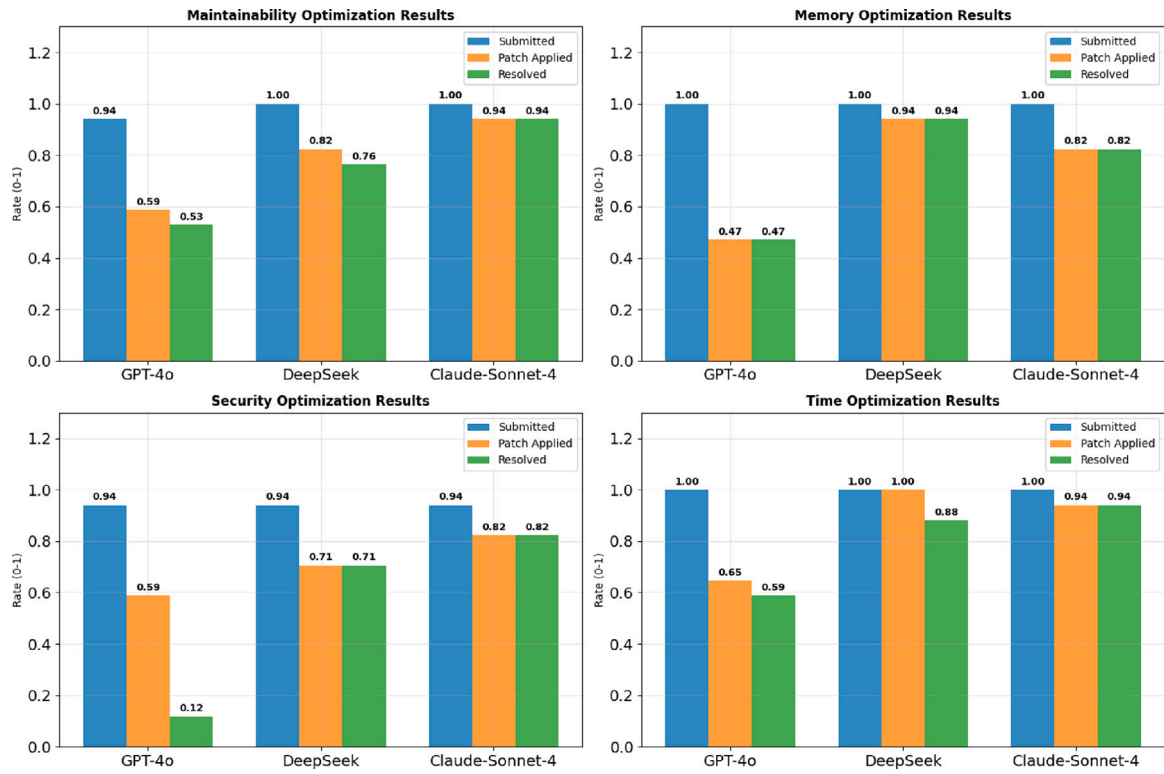


Fig. 9. Patch generation results under different prompt strategies optimized for different NFQCs. Results are shown for three models across the selected metrics: (a) maintainability, (b) memory, (c) security, and (d) test runtime. For each metric, the bars illustrate the rate of submitted patches, the PSA, and the resolved rate. The total number is 17.

Table 6

Results of CodeQL analysis on generated patches in terms of maintainability and security after the first stage. The table presents the number of rules triggered for each severity level across the generated patches and the gold patches. The severity level follows the order: **Error** > **Warning** > **Recommendation**.

Patch	Avg Added LOC	Maintainability			Security		
		E	W	R	E	W	R
Gold-patch	2.65	0	0	30	2	0	0
GPT-4o	16.71	0	2	423	4	0	0
DeepSeek	2.35	0	2	420	6	0	0
Claude-Sonnet-4	5.94	0	2	414	4	0	0

Table 7

Test runtime (in seconds) and memory usage (in MB) of unoptimized patches generated by three LLMs compared with the gold patches. For each model, the mean, standard deviation, minimum, and maximum values are reported across all 17 instances from the common subset.

Model	Test runtime (s)				Memory (MB)			
	mean	std	min	max	mean	std	min	max
Gold-patch	14.26	12.29	3.51	46.28	23.50	3.84	20.04	37.25
GPT-4o	23.27	19.75	5.64	68.77	44.54	25.79	8.79	111.09
DeepSeek	17.68	14.09	4.42	53.67	24.99	7.22	9.25	39.25
Claude-Sonnet-4	16.72	14.72	4.39	51.21	27.27	9.91	8.20	48.61

patches, which are 14.26 s and 23.50 MB. The patches generated by DeepSeek-Reasoner and Claude-Sonnet-4 show similar trends, with maximum test runtimes reaching 53.67 s and 51.21 s, respectively.

RQ 5.2: Effect of NFQC-specific prompt. To address RQ 5.2 and RQ 5.3, we applied NFQC-specific prompts to the patches generated in the first stage and re-evaluated the regenerated patches with respect to both functional correctness and the targeted NFQCs.

On the common instance set, NFQC-specific optimization prompts are associated with reduced functional correctness. This effect is particularly evident for GPT-4o, where the resolved rate under the security optimization decreases to 0.12, corresponding to only two functionally correct patches.

As shown in Table 8, for GPT-4o, within the common instance set, applying the corresponding NFQC-optimization prompts leads to different patterns across metrics. For maintainability, five out of nine instances exhibit fewer issues under the maintainability prompt, while four instances show identical counts, and no instance shows an increase. For runtime, all 10 resolved instances show lower runtime under the runtime optimization prompt. For memory usage, seven out of eight instances exhibit lower memory consumption under the memory prompt, while one instance shows a higher value compared to the baseline. The security optimization only yielded two resolved instances, and we skipped the results.

As shown in Table 9, for DeepSeek, within the common instance set, comparisons between the baseline and the NFQC-specific optimization show limited changes for maintainability and security, and mixed behavior for runtime and memory. For maintainability, one instance shows a reduction under the maintainability optimization, 12 instances remain unchanged, and one instance shows an increase. For security, one instance shows a reduction under the security optimization, 11 instances remain unchanged, and no instances show an increase. For runtime, 12 instances show lower runtime under the runtime optimization, while five instances exhibit higher runtime. Memory usage shows a split pattern. Eight instances exhibit lower memory usage under the memory optimization, while eight instances show higher memory usage.

Table 10 shows the results of Claude-Sonnet-4. Within the common instance set, the application of NFQC-optimization prompts leads to limited changes for maintainability and security, and reductions for runtime and memory in most instances. For maintainability, one instance shows a reduction under the maintainability optimization, 13 instances remain unchanged, and two instances show higher counts.

Table 8

Instance comparison for GPT-4o. M, S, T, and Me denote optimizations for Maintainability, Security, Runtime, and Memory, respectively.

Instance id	Metrics	Baseline	M	S	T	Me
django-11039	Maintainability	10	10	10	14	2
	Security	0	0	0	0	0
	Time	38.81	13.98	20.32	14.06	14.07
	Memory	42.87	21.34	21.32	222.50	21.35
django-11099	Maintainability	2	–	–	0	0
	Security	0	–	–	0	0
	Time	23.36	–	–	12.98	13.19
	Memory	77.76	–	–	174.80	20.62
django-11133	Maintainability	2	–	–	–	–
	Security	0	–	–	–	–
	Time	30.98	–	–	–	–
	Memory	29.91	–	–	–	–
django-12700	Maintainability	2	–	–	–	–
	Security	0	–	–	–	–
	Time	51.15	–	–	–	–
	Memory	51.15	–	–	–	–
django-12983	Maintainability	2	–	2	2	2
	Security	0	–	0	0	0
	Time	28.30	–	12.07	13.15	9.48
	Memory	33.45	–	20.66	181.12	20.85
django-13658	Maintainability	2	0	–	4	–
	Security	0	0	–	0	–
	Time	68.77	46.44	–	49.60	–
	Memory	32.58	22.62	–	182.20	–
django-13710	Maintainability	2	–	–	3	–
	Security	0	–	–	0	–
	Time	16.46	–	–	8.23	–
	Memory	111.09	–	–	195.51	–
django-13933	Maintainability	2	2	–	2	2
	Security	0	0	–	0	0
	Time	5.64	3.65	–	3.65	3.41
	Memory	20.57	18.81	–	177.27	18.84
django-14238	Maintainability	2	2	–	2	2
	Security	0	0	–	0	0
	Time	9.27	6.59	–	6.34	6.16
	Memory	44.94	20.88	–	184.24	20.82
django-14382	Maintainability	2	0	–	2	0
	Security	0	0	–	0	0
	Time	57.03	40.49	–	39.59	39.99
	Memory	31.91	23.78	–	168.48	21.41
django-15851	Maintainability	2	0	–	–	–
	Security	2	0	–	–	–
	Time	9.47	6.20	–	–	–
	Memory	33.84	145.75	–	–	–
django-16046	Maintainability	2	0	–	2	0
	Security	0	0	–	2	0
	Time	7.61	5.85	–	5.91	5.60
	Memory	26.41	20.14	–	179.16	20.13
django-16255	Maintainability	2	2	–	–	–
	Security	0	0	–	–	–
	Time	9.47	5.89	–	–	–
	Memory	28.59	20.74	–	–	–
django-16527	Maintainability	3	–	–	17	–
	Security	0	–	–	0	–
	Time	11.97	–	–	6.64	–
	Memory	85.62	–	–	189.80	–
django-16595	Maintainability	2	–	–	–	–
	Security	2	–	–	–	–
	Time	9.92	–	–	–	–
	Memory	57.53	–	–	–	–
seaborn-3190	Maintainability	0	–	–	–	–
	Security	0	–	–	–	–
	Time	5.71	–	–	–	–
	Memory	8.79	–	–	–	–
xarray-5131	Maintainability	386	181	–	–	181
	Security	0	0	–	–	0
	Time	11.66	8.62	–	–	8.63
	Memory	40.15	16.95	–	–	109.20

Table 9

Instance comparison for DeepSeek. M, S, T, and Me denote optimizations for Maintainability, Security, Runtime, and Memory, respectively.

Instance id	Metrics	Baseline	M	S	T	Me
django-11039	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	14.75	20.66	17.50	20.95	20.98
	Memory	22.74	21.35	21.34	21.50	68.60
django-11099	Maintainability	2	2	8	0	2
	Security	0	0	0	0	0
	Time	11.07	19.80	16.44	20.01	20.13
	Memory	22.32	20.59	20.59	20.94	21.31
django-11133	Maintainability	2	2	–	0	2
	Security	0	0	–	0	0
	Time	10.20	19.43	–	19.65	19.80
	Memory	21.21	20.21	–	20.54	20.57
django-12700	Maintainability	2	2	2	2	3
	Security	0	0	0	0	0
	Time	14.76	22.63	19.16	22.85	23.07
	Memory	22.90	21.71	21.70	26.43	30.38
django-12983	Maintainability	2	3	11	2	–
	Security	0	0	0	0	–
	Time	10.01	9.80	9.68	12.33	–
	Memory	39.25	66.18	20.63	20.64	–
django-13658	Maintainability	2	–	4	0	2
	Security	0	–	0	0	0
	Time	53.67	–	47.11	49.30	46.30
	Memory	24.20	–	22.62	22.75	65.63
django-13710	Maintainability	2	–	2	2	2
	Security	0	–	0	0	0
	Time	22.61	–	8.64	8.66	8.58
	Memory	33.37	–	21.19	21.19	30.49
django-13933	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	4.42	3.49	3.58	3.83	3.67
	Memory	28.74	18.82	18.82	18.96	20.26
django-14238	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	14.39	6.39	6.43	7.03	6.36
	Memory	36.42	20.80	20.80	136.30	24.67
django-14382	Maintainability	2	2	2	0	2
	Security	0	0	0	0	0
	Time	52.11	40.06	41.59	40.25	40.68
	Memory	22.54	21.42	21.41	21.59	21.95
django-15851	Maintainability	2	2	–	0	2
	Security	2	7	–	0	0
	Time	12.05	6.18	–	6.15	6.17
	Memory	21.51	20.42	–	20.52	20.67
django-16046	Maintainability	2	2	2	0	2
	Security	2	2	2	2	2
	Time	10.91	5.75	5.95	5.77	6.00
	Memory	21.23	20.14	20.14	20.16	26.15
django-16255	Maintainability	2	2	–	2	2
	Security	2	2	–	2	2
	Time	13.00	5.92	–	6.06	5.91
	Memory	21.98	20.75	–	20.75	25.10
django-16527	Maintainability	2	2	2	–	2
	Security	0	0	0	–	0
	Time	10.33	6.52	6.85	–	6.66
	Memory	33.24	23.45	23.42	–	37.63
django-16595	Maintainability	2	–	3	0	2
	Security	2	–	1	0	0
	Time	18.38	–	5.86	5.96	5.59
	Memory	24.17	–	22.12	35.02	22.66
seaborn-3190	Maintainability	0	–	–	–	0
	Security	0	–	–	–	0
	Time	5.93	–	–	–	4.49
	Memory	9.25	–	–	–	62.25
xarray-5131	Maintainability	392	183	–	183	183
	Security	0	0	–	0	0
	Time	21.92	8.56	–	8.53	9.35
	Memory	19.78	16.98	–	17.02	84.03

For security, one instance shows a reduction, and one instance shows an increase under the security optimization, while 12 instances remain unchanged. The median security count remains zero for both baseline and security optimization. For runtime, 14 instances show lower runtime under the runtime optimization, while two instances exhibit higher runtime. For memory usage, 14 instances exhibit lower values under the memory optimization, and no instances show higher memory usage compared to the baseline.

Fig. 10 shows the distribution of test runtime and memory usage across different optimization prompts for three models, with gold and baseline patches included as reference points. Across all three models, the runtime distributions show noticeable variation across optimization prompts and individual instances. For GPT-4o and DeepSeek-Reasoner, median test runtimes differ across optimization settings, with some prompts associated with lower medians than the baseline and others showing higher medians. However, the IQRs (Interquartile Range) overlap with those of the gold and baseline patches in all cases, indicating that runtime differences are not uniform across instances. For Claude-Sonnet-4, median test runtimes across optimization prompts remain close to the gold reference, while the baseline setting exhibits greater variability. Outliers are observed under most settings for all models, highlighting the presence of instances with notably higher test runtimes.

Memory usage distributions similarly exhibit overlap across gold, baseline, and optimization-specific prompts for all models. GPT-4o shows larger variability in memory usage, particularly under the baseline and security prompts, whereas the maintainability and memory prompts yield median values closer to the gold patches. For DeepSeek-Reasoner, memory optimization results in increased dispersion, while maintainability and time optimization yield some outliers with higher memory usage. For Claude-Sonnet-4, memory usage remains relatively stable across all optimization settings, with medians closely aligned with the gold patches and limited dispersion, aside from a small number of outliers.

We further analyzed instances where the generated patches under the time optimization setting achieve lower test runtime than the gold patches (10.30 s and 22.52 MB), including *django-16046* and *django-16527*. For these instances, the generated patches introduce simpler and more direct changes that reduce runtime while increasing memory usage. The gold patches, in contrast, make use of existing methods or functions of the project, which results in higher runtime but lower memory usage. This difference in implementation helps explain why the generated patches outperform the gold patches in test runtime for these cases.

RQ 5.3: Trade-offs across NFQCs. According to the results reported in Tables 8–10, for some instances, changes in one NFQC coincide with changes in others, indicating interactions and trade-offs between different NFQCs.

For GPT-4o, although maintainability optimization does not reduce maintainability issues for all generated patches, the patches produced under maintainability optimization exhibit lower runtime and memory usage than the baseline patches for most instances, with the exception of *django-15851*. Runtime optimization tends to degrade the other three NFQCs, especially for memory usage. For example, in the case of *django-13933*, runtime optimization results in a rise from 20.57 MB to 177.27 MB in memory usage compared to the baseline generated patches. Compared with runtime optimization, memory optimization improves the runtime and memory usage of eight resolved instances, but increases the memory usage of *xarray-5131*.

For DeepSeek-Reasoner, maintainability and security optimizations lead to increases in maintainability and security issue counts in a small number of instances. Notably, the *xarray-5131* shows improvements in maintainability under the three NFQC optimizations except security. Runtime optimization produces more mixed effects. It improves maintainability, security, and memory usage for the majority of instances, but a few cases (e.g., *django-14238*) exhibit clear regressions.

Claude-Sonnet-4 exhibits a similar pattern. Maintainability optimization leads to increased maintainability issue counts in a small number of instances (*django-16527* and *django-11133*), and it also results in a substantial increase in memory usage for *django-16595*.

Table 11 shows the results of our statistical analysis using the Wilcoxon signed-rank test. The results show statistically significant and practically meaningful improvements primarily for runtime and memory metrics.

For GPT-4o, runtime improvements are statistically significant across Maintainability, Time and Memory optimization settings ($p = 0.0020, r = 0.979$), and time optimization yields a significant increase in memory usage ($p = 0.0010, r = 0.99$). No maintainability optimization reaches statistical significance, although large effect sizes are observed for some settings. No statistically significant effects are observed for security-related metrics.

For DeepSeek, statistically significant improvements are observed for maintainability under the time optimization ($p = 0.0039, r = 0.70$) and for memory under the security optimization ($p = 0.0005, r = 1.00$). Runtime improvements show a large effect size but do not reach conventional significance thresholds ($p = 0.0640, r = 0.54$), and no statistically significant effects are detected for security-related metrics.

For Claude-Sonnet-4, statistically significant improvements are observed for runtime under the time optimization ($p = 0.0042, r = 0.72$) and for memory under the security, time, and memory optimizations ($p = 0.0001, r = 1.00$). In contrast, maintainability and security optimizations do not yield statistically significant differences.

5.4. Discussion of experiment results

The results demonstrate that current code LLMs face substantial limitations when addressing complex engineering tasks. By simulating how developers realistically use LLMs in their workflows, our evaluation provides insights into their actual applicability in software development. As discussed in Section 5.3, many of the agent retries were spent on resolving environment and command issues rather than the patch logic. These issues reveal a weakness in environment configuration, dependency resolution, and command invocation. In an agentic and robust system, the ability to use tools effectively is an integral part of the overall capability; therefore, such failures indicate limitations at the process level, even when the code generation itself is adequate. Moreover, in the final stage, many patches failed to apply due to context mismatches (see Table 12 and Fig. 11), highlighting persistent difficulties in adhering to the unified diff format despite clear instructions in the prompts.

The comparison across models reveals different patterns. GPT-4o achieved the highest PSA rate in the first evaluation, yet its resolved rate was the lowest, suggesting that many of its patches were syntactically valid but functionally ineffective. After introducing NFQC optimizations, both its application and resolution rates declined, showing limited robustness in balancing functional correctness and NFQCs. In contrast, Claude-Sonnet-4 exhibited a lower PSA in the first evaluation but achieved a higher resolved rate. After the NFQC optimization, Claude-Sonnet-4 achieved the highest PSA and resolved rates under maintainability, security, and runtime optimization, while DeepSeek-Reasoner performed best under memory optimization, with both PSA and resolved rate reaching 94%. These results suggest a specialization of strengths among models across different optimization objectives, rather than a single model outperforming others universally.

A further challenge lies in the lack of NFQC awareness. Under baseline prompts, all models produced patches that were inferior to the gold patches in maintainability, security, runtime, and memory usage. This suggests that agent-based generation pipelines, while capable of producing functionally correct patches, implicitly prioritize “passing the test” rather than “passing with quality.” Such a lack of NFQC awareness constitutes a fundamental barrier to the integration of LLMs into automated workflows. It is not simply that models are unaware

Table 10

Instance comparison for Claude-Sonnet-4. M, S, T, and Me denote optimizations for Maintainability, Security, Runtime, and Memory, respectively.

Instance id	Metrics	Baseline	M	S	T	Me
django-11039	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	12.74	17.34	13.93	17.27	14.09
	Memory	24.40	21.34	21.36	21.35	21.34
django-11099	Maintainability	2	–	2	2	2
	Security	0	–	0	0	0
	Time	30.42	–	13.01	16.43	13.15
	Memory	29.11	–	20.58	20.59	20.59
django-11133	Maintainability	2	6	–	–	–
	Security	0	0	–	–	–
	Time	34.10	15.95	–	–	–
	Memory	22.81	20.21	–	–	–
django-12700	Maintainability	2	2	–	2	–
	Security	0	4	–	0	–
	Time	26.88	18.81	–	19.12	–
	Memory	24.31	21.71	–	21.71	–
django-12983	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	9.14	9.49	13.12	9.45	13.05
	Memory	24.57	20.64	20.64	20.66	20.64
django-13658	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	51.21	46.22	49.63	46.14	49.79
	Memory	25.50	22.63	22.65	22.64	22.88
django-13710	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	12.23	8.12	8.09	8.21	8.01
	Memory	31.60	21.20	21.20	21.20	21.32
django-13933	Maintainability	2	2	2	2	2
	Security	0	0	0	0	0
	Time	4.39	3.43	3.52	3.49	3.54
	Memory	20.27	18.82	18.82	18.82	18.82
django-14238	Maintainability	2	2	2	2	–
	Security	0	0	0	0	–
	Time	7.69	6.24	6.25	6.23	–
	Memory	33.86	20.83	20.80	21.02	–
django-14382	Maintainability	2	2	2	2	10
	Security	0	0	0	0	0
	Time	43.74	40.03	39.94	39.91	40.46
	Memory	26.14	21.61	21.43	21.43	21.44
django-15851	Maintainability	2	2	2	2	2
	Security	0	0	0	2	2
	Time	6.92	6.08	5.91	6.03	5.98
	Memory	24.88	20.41	20.41	20.43	20.42
django-16046	Maintainability	2	2	13	5	2
	Security	0	0	0	0	0
	Time	6.95	5.80	5.66	5.81	5.78
	Memory	30.76	20.13	20.13	20.15	20.14
django-16255	Maintainability	2	2	9	2	2
	Security	0	2	2	0	2
	Time	7.05	5.88	5.99	5.93	6.05
	Memory	22.27	20.80	20.74	20.75	20.75
django-16527	Maintainability	3	44	2	2	2
	Security	0	2	0	0	2
	Time	9.35	6.49	6.39	6.43	6.55
	Memory	48.61	23.45	23.42	23.42	23.44
django-16595	Maintainability	2	2	2	2	2
	Security	2	2	0	0	0
	Time	8.10	5.53	5.53	5.49	5.56
	Memory	48.58	145.16	22.12	22.12	22.12
seaborn-3190	Maintainability	0	0	–	0	0
	Security	0	0	–	0	0
	Time	4.54	4.68	–	4.10	4.05
	Memory	8.20	146.84	–	8.03	8.03
xarray-5131	Maintainability	386	181	181	181	181
	Security	0	0	0	0	0
	Time	8.71	8.39	8.42	8.45	8.27
	Memory	17.69	17.06	17.02	17.09	16.95

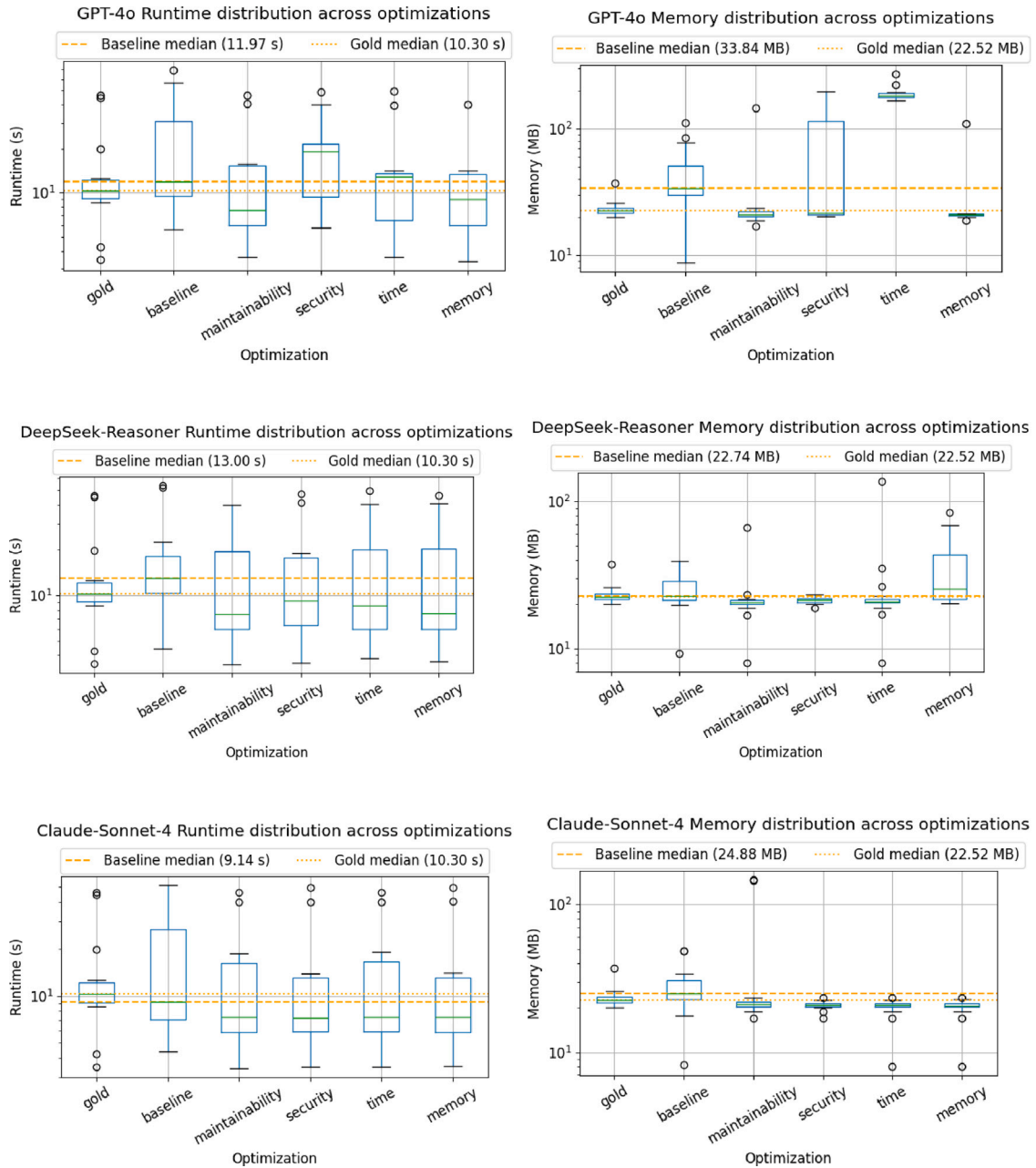


Fig. 10. Boxplots of test runtime (s) and memory usage (MB) on a logarithmic scale for patches generated by GPT-4o, DeepSeek-Reasoner, and Claude-Sonnet-4, compared with the gold and baseline patches. The **baseline** represents the results of the unoptimized generated patches of each model. The boxes summarize the distribution across resolved instances, with dashed and dotted lines indicating the median values of the baseline evaluation and gold patches, respectively.

of best practices; their decision mechanisms do not treat NFQCs as primary objectives. Since LLMs are trained to predict the most likely next token, they capture superficial patterns of code rather than underlying engineering principles. Without an intrinsic cost function for code quality, all test-passing solutions are treated as equivalent. As a result, directly applying these patches risks introducing technical debt: the code may be functionally correct but structurally fragile and suboptimal. This limitation reflects the current model architectures and training paradigms rather than implementation errors.

Our baseline analysis showed that LLM-generated patches triggered more maintainability rules than the gold patches. For example, in Table 6, GPT-4o triggers 423 maintainability recommendations compared to only 30 in the gold patch. Results from DeepSeek-Reasoner and Claude-Sonnet-4 reinforced this pattern. Additionally, the maintainability issues in gold patches are *unused-global-variable*, but most of the

maintainability issues in generated code are *cyclic-import*. The gold patches are typically written with awareness of the existing project structure and therefore tend to reuse established modules and dependency patterns. As a result, their maintainability issues are often localized, such as *unused-global-variable*. In contrast, generated patches often resolve defects in a more self-contained manner, introducing or reorganizing imports without fully considering the global dependency graph. In large Python projects, this can easily lead to *cyclic-import* issues, which substantially increase the number of maintainability warnings reported by CodeQL. This observation also highlights a broader challenge faced by current LLM-based software development. Even when models are allowed to access and explore the entire code repository through prompting, they may still produce errors that stem from an incomplete understanding of the project as a whole. Such errors suggest that current models lack robust project-level awareness, and future

```

INFO - Creating container for django__django-13710...
INFO - Container for django__django-13710 created...
INFO - Container for django__django-13710 started...
INFO - Intermediate patch for django__django-13710 written to logs
      now applying to container...
INFO - Failed to apply patch to container: git apply --verbose
INFO - Failed to apply patch to container: git apply --verbose --
      reject
INFO - Failed to apply patch to container: patch --batch --fuzz=5 -
      p1 -i
INFO - >>>> Patch Apply Failed:
patch: **** malformed patch at line 15:      def media(self):
patching file django/contrib/admin/options.py

```

Fig. 11. Example of malformed patch error during patch application.

Table 11

Significance Test Results and Effect Sizes (r) across Models. We report optimization strategies that achieve both statistical significance ($p < 0.05$) and large effect sizes ($r > 0.5$). For each metric, we additionally report the minimum p -value and the maximum effect size observed across optimization strategies. – indicates that no optimization met the reporting criteria for the corresponding metric. The best results are highlighted in bold.

Model	Metric	Optimization	p -value	Effect(r)
GPT-4o	Maintainability	–	0.0625	0.659
	Time	M/T/Me	0.0020	0.979
	Memory	T	0.0020	0.979
	Security	–	1.0000	0.000
DeepSeek	Maintainability	T	0.0039	0.700
	Time	–	0.0640	0.535
	Memory	S	0.0005	1.000
	Security	–	0.1573	0.354
Sonnet	Maintainability	–	0.5930	0.143
	Time	T	0.0042	0.716
	Memory	S/T/Me	0.0001	1.000
	Security	–	0.1025	0.408

Table 12

Number of malformed patches generated by each model under different optimization prompts (out of 17 patches per optimization). A malformed patch refers to a generated patch that could not be successfully applied to the repository or failed to comply with the unified diff format.

Model/Optimization	Maintainability	Security	Time	Memory
GPT-4o	3	4	3	7
DeepSeek-Reasoner	2	0	0	0
Claude-Sonnet 4	0	1	1	3

research should place greater emphasis on supporting holistic reasoning over large and evolving codebases.

The NFQC optimization results also revealed trade-offs between different NFQCs. We observed interactions between runtime and memory usage in some instances, where improvement in runtime is at the cost of memory usage. More critically, under the optimization prompts used in this study, we observed negative optimization, where models degraded the very dimension they were instructed to improve. These results indicate that optimizing a single NFQC through prompts does not guarantee improvements in isolation. Instead, changes introduced in order to optimize one dimension may interact with other NFQCs or, in some cases, negatively affect the targeted metric itself. This suggests that current NFQC optimization primarily operates through local code transformations that are sensitive to the surrounding code context, rather than through a stable understanding of cross-metric constraints. As a result, NFQC optimization cannot be treated as a reliable, standalone procedure. Any optimization attempt should be accompanied by targeted validation, as improvements observed for one

NFQC or one instance cannot be assumed to generalize across metrics or across instances.

Finally, two particular instances stand out in our results. *xarray-5131* showed consistent improvements in maintainability and security across models and optimizations, suggesting that instances with severe initial deficiencies and relatively lenient functional constraints allow for modifications without breaking tests. By contrast, *seaborn-3190* illustrated the inherent conflict among NFQCs. Across several optimizations, functional correctness was compromised, suggesting that its code was in a fragile equilibrium where any modification risked undermining both security and functionality.

6. Threats to validity

This section presents the potential issues that may affect the validity and reproducibility of our findings.

6.1. Internal validity

For the literature review, our search strategy followed key elements of the PRISMA guidelines (Page et al., 2021), but there is still a risk that relevant studies have been missed due to database coverage limitations or the keywords. To mitigate the risk, we applied both forward and backward snowballing and found several relevant studies.

For the workshops, two authors independently took notes during the workshops; individual interpretations may have introduced subjective bias and affected the objectivity of the notes. To mitigate this, we conducted joint discussions after each workshop to reconcile the two records, ensuring a consistent interpretation of the practitioners' views.

In our experiments, we combined the prompts provided in the SWE-bench Lite benchmark with the CodeQL reports to design NFQC optimization prompts. This procedure mitigates subjective bias to some extent, but prompt sensitivity remains a threat. Minor changes in the prompts could affect LLM behavior and result in different generated patches. In addition, different hyperparameter settings, such as temperature, may influence the stochasticity and diversity of model outputs.

Second, our experiments were conducted on a single workstation (AMD Ryzen 7, 16 GB RAM, Ubuntu 22.04). Although the evaluation environment was kept consistent across all models and stages, background system loads could have introduced small variations in runtime and memory measurements. These potential fluctuations were not explicitly recorded.

Third, we used three LLMs in the experiment. But the models update frequently during the experiment period, which could lead to model drift and different behaviors in the future. This is a known limitation when evaluating LLMs.

Finally, to ensure comparability across models, we restricted our analysis to instances successfully solved by three models. However, it may create a bias towards simpler tasks, potentially skewing the observed trade-offs among NFQCs.

6.2. Construct validity

For the literature review, we mapped diverse quality characteristics from different studies to the ISO/IEC 25010 quality model; there is a risk that we may not always understand and reflect the authors' original intent during the mapping.

Individual understanding of concepts such as readability and security by participants might also have varied, leading to potential inconsistencies in the way in which some discussions were understood. To mitigate this, we reviewed and discussed our notes to ensure a consistent understanding of their perspectives.

The metrics used in the experiment represent practical but partial approximations of NFQCs. In particular, we rely on CodeQL findings to operationalize maintainability and security of generated patches. As a static analysis tool, CodeQL provides reproducible indicators of certain code-level quality issues, but it captures only a subset of the underlying quality dimensions. To partially mitigate this limitation, we additionally report patch size and conduct a selective qualitative human review of representative generated patches, which provides complementary perspectives on code changes and helps contextualize the automated results. Nevertheless, we acknowledge that CodeQL-based measurements cannot fully reflect broader aspects of code quality, such as readability, architectural coherence, or developer-perceived maintainability. Future work should therefore incorporate more comprehensive evaluation approaches, including additional static and dynamic analysis tools as well as more extensive human-centered assessment, to obtain a more holistic view of non-functional code quality.

6.3. External validity

Most of the workshop participants are Software Center companies, and there is a risk that their perspectives may not fully represent the broader software industry.

The generalizability of our findings is constrained by both the dataset and the experimental design. First, our study is based on SWE-bench Lite, which consists of real-world issue-resolution tasks. This benchmark is well-suited for studying NFQCs in repository-level patches, but the observed patterns may not extend to other generation settings, such as feature implementation or class-level synthesis (Du et al., 2024; Li et al., 2025). At the same time, compared to synthetic benchmarks (e.g., HumanEval), SWE-bench Lite requires multi-file navigation, dependency analysis, and interaction with existing codebases, which more closely reflect professional software maintenance scenarios. Additionally, the instances included in SWE-bench are skewed toward framework-based Python repositories, particularly Django (127 Django out of 300 instances). This framework dominance influences both the nature of the vulnerabilities and the manifestation of NFQCs, as many security features are abstracted away by the framework, and maintainability challenges differ from those in lower-level codebases.

Second, the analysis is restricted to instances that were successfully resolved by all evaluated models. This restriction biases the sample toward tasks that are jointly solvable and therefore likely easier than the full benchmark. As a result, the findings should be interpreted as characterizing non-functional quality behavior under relatively favorable conditions. Since quality degradation and trade-offs are more likely to emerge in complex or borderline cases, the reported results may underestimate the extent of NFQC issues that arise in harder tasks.

Third, the final dataset contains 17 instances. While limited in size, this selection enables detailed qualitative inspection, including manual review and patch-size analysis, which would be difficult to apply consistently at larger scales. The study, therefore, emphasizes analytical depth over coverage.

Finally, although the specific models evaluated reflect a snapshot of rapidly evolving code generation systems, the observed patterns, such as tensions between functional success and structural maintainability, are likely indicative of broader tendencies in current LLM-based code generation approaches. Extending the analysis to larger and more diverse task sets, including instances not jointly solved by all models, remains an important direction for future work.

7. Conclusion and future work

7.1. Conclusion

Our study examined the performance of LLM-generated code with respect to NFQCs in real-world software engineering tasks. By combining evidence from a literature review, industry workshops, and empirical experiments, we provide a multi-perspective view of how NFQCs are currently conceptualized, prioritized, and manifested in LLM-assisted code generation.

The literature review shows that existing research has mainly focused on security, performance efficiency, maintainability, and reliability, while other NFQCs have received comparatively little attention. At the same time, different studies often employ inconsistent terminology for the same NFQC, which hampers cross-study comparisons and highlights the absence of a unified framework.

Insights from workshops highlight the differences between academic and industrial perspectives. While academic research tends to emphasize security and performance efficiency, practitioners show stronger concern for maintainability, particularly in the context of large software systems and long-term maintenance.

Our experiments explore the interactions between NFQCs in generated code, demonstrating the instability of optimizing NFQCs through single-prompt code generation in practical software engineering settings, and showing that applying LLM-generated patches to projects without validation entails the risk of introducing and amplifying technical debt.

Taken together, these findings illustrate existing limitations and trade-offs in how NFQCs are handled in current LLM-based code generation workflows. Rather than drawing general conclusions about LLM capabilities in all settings, this study highlights the importance of aligning evaluation frameworks, optimization strategies, and validation practices with the quality concerns that arise in real-world software development.

7.2. Future work

To move the generated code from “passing the tests” to “passing with quality”, future work should connect evaluation, modeling, and tooling. In future work, we are going to integrate strong verification mechanisms into the generation pipeline. We will embed static analysis and automated detectors into generation and optimization so that NFQCs are monitored in real time, used as gates for candidate patches, and returned as structured feedback for revision. By addressing NFQCs before deployment, we aim to reduce technical debt accumulation and improve the reliability of LLM-generated code in software systems.

CRedit authorship contribution statement

Xin Sun: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Daniel Ståhl:** Writing – review & editing, Supervision, Methodology, Investigation, Conceptualization. **Kristian Sandahl:** Writing – review & editing, Supervision, Methodology, Investigation, Conceptualization. **Christoph Kessler:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was funded by the Software Center project 61 and the Vinnova Competence Center for Continuous Digitalization 2023-00546. The authors are indebted to Willem Meijer for fruitful discussions.

Data availability

Data will be made available on request.

References

- Asare, O., Nagappan, M., Asokan, N., 2023b. Is GitHub's copilot as bad as humans at introducing vulnerabilities in code? *Empir. Softw. Eng.* 28 (6), 129. <http://dx.doi.org/10.1007/s10664-023-10380-1>.
- Bolt, 2024. Bolt – your AI-powered development agent. URL <https://bolt.new>. (Accessed 13 May 2025).
- Börstler, J., Bennin, K.E., Hooshangi, S., Jeuring, J., Keuning, H., Kleiner, C., MacKellar, B., Duran, R., Störle, H., Toll, D., van Assema, J., 2023. Developers talking about code quality. *Empir. Softw. Eng.* 28 (6), 128. <http://dx.doi.org/10.1007/s10664-023-10381-0>.
- Chen, X., Gao, C., Chen, C., Zhang, G., Liu, Y., 2025. An empirical study on challenges for LLM application developers. *ACM Trans. Softw. Eng. Methodol.* <http://dx.doi.org/10.1145/3715007>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F.P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W.H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A.N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., Zaremba, W., 2021. Evaluating large language models trained on code. [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- Coignon, T., Quinton, C., Rouvoy, R., 2024b. A performance study of LLM-generated code on LeetCode. In: *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. EASE 2024, Salerno, Italy, June 18–21, 2024, ACM, pp. 79–89. <http://dx.doi.org/10.1145/3661167.3661221>.
- Cursor, 2024. Cursor: The AI coding assistant. URL <https://www.cursor.com>. (Accessed 13 May 2025).
- Della Porta, A., Lambiase, S., Palomba, F., 2025. Do prompt patterns affect code quality? A first empirical assessment of ChatGPT-generated code. In: *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*. In: EASE '25, Association for Computing Machinery, New York, NY, USA, pp. 181–192. <http://dx.doi.org/10.1145/3756681.3756938>.
- Deng, L., Jiang, Z., Cao, J., Pradel, M., Liu, Z., 2025. Nocode-bench: A benchmark for evaluating natural language-driven feature addition. <http://dx.doi.org/10.48550/ARXIV.2507.18130>, [arXiv:2507.18130](https://arxiv.org/abs/2507.18130).
- Du, X., Liu, M., Wang, K., Wang, H., Liu, J., Chen, Y., Feng, J., Sha, C., Peng, X., Lou, Y., 2024. Evaluating large language models in class-level code generation. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. ICSE 2024, Lisbon, Portugal, April 14–20, 2024, ACM, pp. 81:1–81:13. <http://dx.doi.org/10.1145/3597503.3639219>.
- Elgedawy, R., Sadik, J., Dutta, S., Gautam, A., Georgiou, K., Gholamrezae, F., Ji, F., Lim, K., Liu, Q., Ruoti, S., 2024. Occasionally secure: a comparative analysis of code generation assistants. <http://dx.doi.org/10.48550/arXiv.2402.00689>, [arXiv:2402.00689](https://arxiv.org/abs/2402.00689).
- Eltabakh, T.M., Nabil Soudi, N., Shawky, D., 2024. Quality of AI-generated vs. Human-generated code. In: *2024 34th International Conference on Computer Theory and Applications*. ICCTA, pp. 200–205. <http://dx.doi.org/10.1109/ICCTA64612.2024.10974782>.
- Fu, Y., Liang, P., Tahir, A., Li, Z., Shahin, M., Yu, J., Chen, J., 2025a. Security weaknesses of copilot-generated code in GitHub projects: an empirical study. *ACM Trans. Softw. Eng. Methodol.* <http://dx.doi.org/10.1145/3716848>.
- GitHub, 2025. Github copilot: AI code completion tool. URL <https://github.com/features/copilot>. (Accessed 20 April 2025).
- Hemel, Z., Kats, L.C.L., Visser, E., 2008. Code generation by model transformation. In: Vallecillo, A., Gray, J., Pierantonio, A. (Eds.), *Theory and Practice of Model Transformations*. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 183–198.
- Huang, D., Dai, J., Weng, H., Wu, P., Qing, Y., Cui, H., Guo, Z., Zhang, J.M., 2024a. EffiLearner: enhancing efficiency of generated code via self-optimization. *Adv. Neural Inf. Process. Syst.* 37, 84482–84522.
- Huang, D., Qing, Y., Shang, W., Cui, H., Zhang, J., 2024b. Effibench: Benchmarking the efficiency of automatically generated code. In: Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J.M., Zhang, C. (Eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024*. NeurIPS 2024, Vancouver, BC, Canada, December 10–15, 2024, URL http://papers.nips.cc/paper_files/paper/2024/hash/15807b6e09d691fe5e96cdced6d7b80-Abstract-Datasets_and_Benchmarks_Track.html.
- Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Dang, K., Yang, A., Men, R., Huang, F., Ren, X., Ren, X., Zhou, J., Lin, J., 2024. Qwen2.5-coder technical report. <http://dx.doi.org/10.48550/ARXIV.2409.12186>, [arXiv:2409.12186](https://arxiv.org/abs/2409.12186).
- International Organization for Standardization, 2023. ISO/IEC 25010:2023 systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuARE) — Product quality model. <https://www.iso.org/standard/78176.html>. (Accessed 19 April 2025).
- Jiang, A.Q., Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, D.S., Casas, D.d., Bressand, F., Lengyel, G., Lample, G., Saulnier, L., Lavaud, L.R., Lachaux, M., Stock, P., Scao, T.L., Lavril, T., Wang, T., Lacroix, T., Sayed, W.E., 2023. Mistral 7B. <http://dx.doi.org/10.48550/ARXIV.2310.06825>, [arXiv:2310.06825](https://arxiv.org/abs/2310.06825).
- Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.R., 2024. SWE-bench: Can language models resolve real-world github issues? In: *The Twelfth International Conference on Learning Representations*. ICLR 2024, Vienna, Austria, May 7–11, 2024, OpenReview.net, URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Kang, D., Seo, K.J., Kim, T., 2024. Revisiting the impact of pursuing modularity for code generation. In: Al-Onaizan, Y., Bansal, M., Chen, Y. (Eds.), *Findings of the Association for Computational Linguistics*. EMNLP 2024, Miami, Florida, USA, November 12–16, 2024, Association for Computational Linguistics, pp. 11561–11571, URL <https://aclanthology.org/2024.findings-emnlp.676>.
- Khojah, R., de Oliveira Neto, F.G., Mohamad, M., Leitner, P., 2025. The impact of prompt programming on function-level code generation. *IEEE Trans. Softw. Eng.* 51 (8), 2381–2395. <http://dx.doi.org/10.1109/TSE.2025.3587794>.
- Khoury, R., Avila, A.R., Brunelle, J., Camara, B.M., 2023. How secure is code generated by ChatGPT? In: *IEEE International Conference on Systems, Man, and Cybernetics*. SMC 2023, Honolulu, Oahu, HI, USA, October 1–4, 2023, IEEE, pp. 2445–2451. <http://dx.doi.org/10.1109/SMC53992.2023.10394237>.
- Klemmer, J.H., Horstmann, S.A., Patnaik, N., Ludden, C., Burton, C., Powers, C., Massacci, F., Rahman, A., Votipka, D., Lipford, H.R., Rashid, A., Naiakshina, A., Fahl, S., 2024. Using AI assistants in software development: a qualitative study on security practices and concerns. In: *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. In: CCS '24, Association for Computing Machinery, New York, NY, USA, pp. 2726–2740. <http://dx.doi.org/10.1145/3658644.3690283>.
- Le, H., Chen, H., Saha, A., Gokul, A., Sahoo, D., Joty, S., 2024. Codechain: towards modular code generation through chain of self-revisions with representative sub-modules. In: *ICLR2024*.
- Li, R., Allal, L.B., Zi, Y., Muennighoff, N., Kocetkov, D., Mou, C., et al., 2023. Starcoder: may the source be with you! *Trans. Mach. Learn. Res.* 2023, URL <https://openreview.net/forum?id=KoFOg41haE>.
- Li, Y., Choi, D.H., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., et al., 2022. Competition-level code generation with AlphaCode. *Science* 378 (6624), 1092–1097. <http://dx.doi.org/10.1126/science.abq1158>, <https://www.science.org/doi/pdf/10.1126/science.abq1158>, URL <https://www.science.org/doi/abs/10.1126/science.abq1158>.
- Li, W., Zhang, X., Guo, Z., Mao, S., Luo, W., Peng, G., Huang, Y., Wang, H., Li, S., 2025. FEA-bench: A benchmark for evaluating repository-level code generation for feature implementation. In: Che, W., Nabende, J., Shutova, E., Pilehvar, M.T. (Eds.), *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL 2025, Vienna, Austria, July 27–August 1, 2025, Association for Computational Linguistics, pp. 17160–17176, URL <https://aclanthology.org/2025.acl-long.839/>.
- Licorish, S.A., Bajpai, A., Arora, C., Wang, F., Tantithamthavorn, K., 2025. Comparing human and LLM generated code: The jury is still out! <http://dx.doi.org/10.48550/arXiv.2501.16857>, [arXiv:2501.16857](https://arxiv.org/abs/2501.16857).
- Liu, Y., Le-Cong, T., Widyasari, R., Tantithamthavorn, C., Li, L., Le, X.-B.D., Lo, D., 2024a. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *ACM Trans. Softw. Eng. Methodol.* 33 (5), 116:1–116:26. <http://dx.doi.org/10.1145/3643674>.
- Liu, Z., Tang, Y., Luo, X., Zhou, Y., Zhang, L.F., 2024. No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. *IEEE Trans. Softw. Eng.* 50 (6), 1548–1584. <http://dx.doi.org/10.1109/TSE.2024.3392499>.
- Liventsev, V., Grishina, A., Härmä, A., Moonen, L., 2023. Fully autonomous programming with large language models. In: Silva, S., Paquete, L. (Eds.), *Proceedings of the Genetic and Evolutionary Computation Conference*. GECCO 2023, Lisbon, Portugal, July 15–19, 2023, ACM, pp. 1146–1155. <http://dx.doi.org/10.1145/3583131.3590481>.
- Lovable, 2024. Build apps with an AI engineer. URL <https://lovable.dev>. (Accessed 13 May 2025).
- Luo, Z., Xu, C., Zhao, P., Sun, Q., Geng, X., Hu, W., Tao, C., Ma, J., Lin, Q., Jiang, D., 2024. Wizardcoder: Empowering code large language models with evolution. In: *The Twelfth International Conference on Learning Representations*. URL <https://openreview.net/forum?id=UnUwSlgK5W>.
- Manna, Z., Waldinger, R.J., 1971. Toward automatic program synthesis. *Commun. ACM* 14 (3), 151–165. <http://dx.doi.org/10.1145/362566.362568>.
- MITRE, 2024. CWE top 25 most dangerous software weaknesses. URL https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html. (Accessed 13 May 2025).
- Moradi Dakhel, A., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M.C., Jiang, Z.M.J., 2023. GitHub copilot AI pair programmer: Asset or liability? *J. Syst. Softw.* 203, 111734. <http://dx.doi.org/10.1016/j.jss.2023.111734>.

- Moratis, K., Diamantopoulos, T., Nastos, D.-N., Symeonidis, A., 2024. Write me this code: An analysis of ChatGPT quality for producing source code. In: Proceedings of the 21st International Conference on Mining Software Repositories. In: MSR '24, Association for Computing Machinery, New York, NY, USA, pp. 147–151. <http://dx.doi.org/10.1145/3643991.3645070>.
- Natella, R., Liguori, P., Improta, C., Cukic, B., Cotroneo, D., 2024. AI code generators for security: Friend or foe? *IEEE Secur. Priv.* 22 (5), 73–81. <http://dx.doi.org/10.1109/MSEC.2024.3355713>.
- Nguyen, N., Nadi, S., 2022b. An empirical evaluation of GitHub copilot's code suggestions. In: 19th IEEE/ACM International Conference on Mining Software Repositories. MSR 2022, Pittsburgh, PA, USA, May 23–24, 2022, ACM, pp. 1–5. <http://dx.doi.org/10.1145/3524842.3528470>.
- Niu, C., Zhang, T., Li, C., Luo, B., Ng, V., 2024a. On evaluating the efficiency of source code generated by LLMs. In: Lo, D., Xia, X., Penta, M.D., Hu, X. (Eds.), Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering. FORGE 2024, Lisbon, Portugal, 14 April 2024, ACM, pp. 103–107. <http://dx.doi.org/10.1145/3650105.3652295>.
- Ouyang, S., Zhang, J.M., Harman, M., Wang, M., 2024. An empirical study of the non-determinism of ChatGPT in code generation. *ACM Trans. Softw. Eng. Methodol.* 3697010. <http://dx.doi.org/10.1145/3697010>, [arXiv:2308.02828](https://arxiv.org/abs/2308.02828).
- Page, M.J., McKenzie, J.E., Bossuyt, P.M., Boutron, I., Hoffmann, T.C., Mulrow, C.D., Shamseer, L., Tetzlaff, J.M., Akl, E.A., Brennan, S.E., Chou, R., Glanville, J., Grimshaw, J.M., Hróbjartsson, A., Lalu, M.M., Li, T., Loder, E.W., Mayo-Wilson, E., McDonald, S., McGuinness, L.A., Stewart, L.A., Thomas, J., Tricco, A.C., Welch, V.A., Whiting, P., Moher, D., 2021. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ (Clinical Res. Ed.)* 372, <http://dx.doi.org/10.1136/bmj.n71>, [arXiv:https://www.bmj.com/content/372/bmj.n71.full.pdf](https://www.bmj.com/content/372/bmj.n71.full.pdf).
- Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., Karri, R., 2022. Asleep at the keyboard? Assessing the security of GitHub copilot's code contributions. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE Computer Society, pp. 754–768. <http://dx.doi.org/10.1109/SP46214.2022.9833571>.
- Perry, N., Srivastava, M., Kumar, D., Boneh, D., 2023. Do users write more insecure code with AI assistants? In: Meng, W., Jensen, C.D., Cremers, C., Kirda, E. (Eds.), Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. CCS 2023, Copenhagen, Denmark, November 26–30, 2023, ACM, pp. 2785–2799. <http://dx.doi.org/10.1145/3576915.3623157>.
- Ren, X., Ye, X., Zhao, D., Xing, Z., Yang, X., 2024. From misuse to mastery: Enhancing code generation with knowledge-driven AI chaining. In: Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering. In: ASE '23, IEEE Press, Echtermach, Luxembourg, pp. 976–987. <http://dx.doi.org/10.1109/ASE56229.2023.00143>.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Remez, T., Rapi, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Canton-Ferrer, C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G., 2023. Code llama: Open foundation models for code. <http://dx.doi.org/10.48550/ARXIV.2308.12950>, [arXiv:2308.12950](https://arxiv.org/abs/2308.12950).
- Ságodi, Z., Siket, I., Ferenc, R., 2024. Methodology for code synthesis evaluation of LLMs presented by a case study of ChatGPT and copilot. *IEEE Access* 12, 72303–72316. <http://dx.doi.org/10.1109/ACCESS.2024.3403858>.
- Sakib, F.A., Khan, S.H., Karim, A.H.M.R., 2024. Extending the frontier of ChatGPT: Code generation and debugging. In: 2024 International Conference on Electrical, Computer and Energy Technologies. ICECET, pp. 1–6. <http://dx.doi.org/10.1109/ICECET61485.2024.10698405>.
- Shehab, M.A., Wardat, M., Omari, S., Jararweh, Y., 2024. Evaluating large language models for code generation: Assessing accuracy, quality, and performance. In: 2024 2nd International Conference on Foundation and Large Language Models. FLLM, pp. 407–416. <http://dx.doi.org/10.1109/FLLM63129.2024.10852439>.
- Shypula, A., Madaan, A., Zeng, Y., Alon, U., Gardner, J.R., Yang, Y., Hashemi, M., Neubig, G., Ranganathan, P., Bastani, O., Yazdanbakhsh, A., 2024b. Learning performance-improving code edits. In: The Twelfth International Conference on Learning Representations. ICLR 2024, Vienna, Austria, May 7–11, 2024, OpenReview.net, URL <https://openreview.net/forum?id=ix7rLVHXyY>.
- Siddiq, M., Da Silva Santos, J., Devareddy, S., Muller, A., 2024a. Sallm: Security assessment of generated code. In: Proceedings - 2024 39th ACM/IEEE International Conference on Automated Software Engineering Workshops. ASEW 2024, pp. 54–65. <http://dx.doi.org/10.1145/3691621.3694934>.
- Siddiq, M.L., Majumder, S.H., Mim, M.R., Jajodia, S., Santos, J.C.S., 2022. An empirical study of code smells in transformer-based code generation techniques. In: 2022 IEEE 22nd International Working Conference on Source Code Analysis and Manipulation. SCAM, pp. 71–82. <http://dx.doi.org/10.1109/SCAM55253.2022.00014>.
- Siddiq, M., Roney, L., Zhang, J., Santos, J., 2024b. Quality assessment of ChatGPT generated code and their use by developers. In: Proceedings - 2024 IEEE/ACM 21st International Conference on Mining Software Repositories. MSR 2024, pp. 152–156. <http://dx.doi.org/10.1145/3643991.3645071>.
- Siddiq, M.L., Santos, J.C.S., 2022. SecurityEval dataset: mining vulnerability examples to evaluate machine learning-based code generation techniques. In: MSR4P&S 2022, Association for Computing Machinery, New York, NY, USA, pp. 29–33. <http://dx.doi.org/10.1145/3549035.3561184>.
- Singhal, M., Aggarwal, T., Awasthi, A., Natarajan, N., Kanade, A., 2024. NoFunEval: Funny how code LMs falter on requirements beyond functional correctness. <http://dx.doi.org/10.48550/arXiv.2401.15963>, [arXiv:2401.15963](https://arxiv.org/abs/2401.15963).
- Solovyeva, L., Weidmann, S., Castor, F., 2025. AI-powered, but power-hungry? Energy efficiency of LLM-generated code. In: IEEE/ACM Second International Conference on AI Foundation Models and Software Engineering. Forge@ICSE 2025, Ottawa, ON, Canada, April 27–28, 2025, IEEE, pp. 49–60. <http://dx.doi.org/10.1109/FORGE66646.2025.00012>.
- The AI Digest, 2024. Time horizons: How far can we see in AI?. URL <https://theaidigest.org/time-horizons>. (Accessed 12 May 2025).
- Tony, C., Díaz Ferreyra, N.E., Mutas, M., Dhif, S., Scandariato, R., 2025. Prompting techniques for secure code generation: A systematic investigation. *ACM Trans. Softw. Eng. Methodol.* <http://dx.doi.org/10.1145/3722108>.
- Tony, C., Mutas, M., Ferreyra, N.E.D., Scandariato, R., 2023. LLMSecEval: A dataset of natural language prompts for security evaluations. In: 2023 IEEE/ACM 20th International Conference on Mining Software Repositories. MSR, IEEE, Melbourne, Australia, pp. 588–592. <http://dx.doi.org/10.1109/MSR59073.2023.00084>.
- Vartziotis, T., Dellatolas, I., Dasoulas, G., Schmidt, M., Schneider, F., Hoffmann, T., Kotsopoulos, S., Keckeisen, M., 2024. Learn to code sustainably: An empirical study on green code generation. In: LLM4CODE@ICSE. pp. 30–37. <http://dx.doi.org/10.1145/3643795.3648394>.
- Waghjale, S., Veerendranath, V., Wang, Z., Fried, D., 2024. ECCO: can we improve model-generated code efficiency without sacrificing functional correctness? In: Al-Onaizan, Y., Bansal, M., Chen, Y. (Eds.), Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. EMNLP 2024, Miami, FL, USA, November 12–16, 2024, Association for Computational Linguistics, pp. 15362–15376, URL <https://aclanthology.org/2024.emnlp-main.859>.
- Wang, J., Chen, Y., 2023. A review on code generation with LLMs: Application and evaluation. In: 2023 IEEE International Conference on Medical Artificial Intelligence. MedAI, pp. 284–289. <http://dx.doi.org/10.1109/MedAI59581.2023.00044>.
- Weyssow, M., Kamanda, A., Zhou, X., Sahraoui, H., 2025. CodeUltraFeedback: An LLM-as-a-judge dataset for aligning large language models to coding preferences. *ACM Trans. Softw. Eng. Methodol.* <http://dx.doi.org/10.1145/3736407>.
- Windsurf, 2024. Windsurf editor: The AI-native IDE. URL <https://windsurf.com>. (Accessed 13 May 2025).
- Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K.R., Press, O., 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. In: The Thirty-Eighth Annual Conference on Neural Information Processing Systems. URL <https://arxiv.org/abs/2405.15793>.
- Yang, Z., Sun, Z., Yue, T.Z., Devanbu, P., Lo, D., 2024. Robustness, security, privacy, explainability, efficiency, and usability of large language models for code. URL <https://arxiv.org/abs/2403.07506>.
- Yetistiren, B., Özsoy, I., Ayerdem, M., Tüzün, E., 2023. Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub copilot, amazon CodeWhisperer, and ChatGPT. <http://dx.doi.org/10.48550/ARXIV.2304.10778>, [arXiv:2304.10778](https://arxiv.org/abs/2304.10778).
- Zhang, K., Li, J., Li, G., Shi, X., Jin, Z., 2024. CodeAgent: Enhancing code generation with tool-integrated agent systems for real-world repo-level coding challenges. In: Ku, L.-W., Martins, A., Srikumar, V. (Eds.), Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Bangkok, Thailand, pp. 13643–13658. <http://dx.doi.org/10.18653/v1/2024.acl-long.737>, URL <https://aclanthology.org/2024.acl-long.737/>.
- Zhao, J., Sun, Y., Huang, C., Liu, C., Guan, Y., Zeng, Y., Liu, Y., 2025. Towards secure code generation with LLMs: A study on common weakness enumeration. *IEEE Trans. Softw. Eng.* 51 (12), 3507–3523. <http://dx.doi.org/10.1109/TSE.2025.3619281>.
- Zheng, J., Cao, B., Ma, Z., Pan, R., Lin, H., Lu, Y., Han, X., Sun, L., 2024. Beyond correctness: Benchmarking multi-dimensional code generation for large language models. <http://dx.doi.org/10.48550/ARXIV.2407.11470>, [arXiv:2407.11470](https://arxiv.org/abs/2407.11470).
- Zheng, Z., Ning, K., Zhong, Q., Chen, J., Chen, W., Guo, L., Wang, W., Wang, Y., 2024. Towards an understanding of large language models in software engineering tasks. *Empir. Softw. Eng.* 30 (2), 50. <http://dx.doi.org/10.1007/s10664-024-10602-0>.
- Zhong, L., Wang, Z., 2024. Can LLM replace stack overflow? A study on robustness and reliability of large language model code generation. *Proc. the AAAI Conf. Artif. Intell.* 38 (19), 21841–21849. <http://dx.doi.org/10.1609/aaai.v38i19.30185>.
- Zhong, L., Wang, Z., Shang, J., 2024. Debug like a human: A large language model debugger via verifying runtime execution step by step. In: Ku, L., Martins, A., Srikumar, V. (Eds.), Findings of the Association for Computational Linguistics. ACL 2024, Bangkok, Thailand and Virtual Meeting, August 11–16, 2024, Association for Computational Linguistics, pp. 851–870. <http://dx.doi.org/10.18653/V1/2024.FINDINGS-ACL.49>.